EdVisingU AI Ecosystem

THE MASTER BUILD MANUAL

Build your own AI operating system from scratch

Dr. Andre De Freitas (Dr. D)

EdVisingU | DrD Learn | CrediVersity | HireEd Nexus Labs

andre@edvisingu.ca | Version 2.0 | June 2026

edvisingu.com • crediversity.com • gohireed.com • ai.edvisingu.com

SECTION 0: CRITICAL CORRECTIONS - READ FIRST

> **WARNING** Read this entire section before touching anything. Previous documentation incorrectly dismissed OpenJarvis. This section sets the record straight on the full 4-component architecture.

0.1 OpenJarvis - The Top-Level AI OS

github.com/open-jarvis/OpenJarvis is the correct foundation for this system. Previous documentation incorrectly dismissed it. That was wrong - ignore that correction.

OpenJarvis is a Stanford research project (Scaling Intelligence Lab + Hazy Research) for composable, local-first personal AI. It is the top-level orchestrator that sits above everything else in the stack.

What OpenJarvis provides:

- Orchestrator/Operative pattern - Orchestrator breaks complex tasks into subtasks and delegates; Operative executes recurring workflows
- Multi-step reasoning with tool use - web search, calculator, file I/O, code interpreter, retrieval
- MCP server integration - connect any external service
- 7 built-in agent types from simple chat to orchestrated multi-agent workflows
- Backend API server + React frontend dashboard (http://your-vps:5173)
- Preset workflows: morning-digest, deep-research, code-assistant and more

Where it sits in the stack:

- OpenJarvis - top-level AI OS (orchestrates everything)
- hermes-agent - communication gateway (Telegram, Discord, Slack - one of the tools OpenJarvis uses)
- Claude Sonnet 4.6 - primary reasoning model (strategy, writing, decisions)
- Codex / GPT-4o - runs in parallel with Claude for its specific expertise (code generation)
- 25-agent fleet - FastAPI specialist agents called by the orchestrator

Install on Hetzner VPS (full setup in Section 21.10):

```
curl -fsSL https://openjarvis.ai/install.sh | bash
jarvis doctor   # Verify all components healthy
```

> **NOTE** *OpenJarvis is the brain. hermes-agent is the mouth and ears. Claude + Codex are the intelligence. The 25-agent fleet is the hands. Build in that order.*

0.2 Hermes Agent - Architecture Decision (Option 2)

NousResearch/hermes-agent is a REAL, production-grade open-source agent framework with 153k GitHub stars and 24k forks. It is NOT just an LLM model. The original blueprint incorrectly dismissed it.

Architecture Decision: Option 2 - Hermes-agent as Gateway Layer

- hermes-agent handles: Telegram, Discord, Slack, WhatsApp, CLI interface, memory, cron scheduling, MCP integration, multi-model routing
- The 25-agent Docker fleet handles: specialist processing (hermes-content, hermes-finance, hermes-seo, etc.)
- hermes-agent dispatches to the fleet via a custom dispatch skill that makes HTTP calls to the FastAPI router
- Result: battle-tested gateway (not custom code) + fully customized specialist fleet

What hermes-agent gives you for free (no custom code needed):

- Telegram bot gateway - replaces the custom telegram_bot.py entirely
- Built-in memory system with FTS5 search across sessions
- Skills system - define specialist routing as skills
- Cron scheduler with Telegram/Discord delivery
- MCP server integration
- Multi-model support: Claude, Gemini, OpenRouter (200+ models), Ollama
- Subagent spawning and parallel workstreams

> **WARNING** Do NOT build a custom Telegram bot. Use hermes-agent gateway instead - it is already built, tested, and maintained.

0.3 Hardware Reality Check

The HP EliteBook 850 G7 has no NVIDIA GPU and no CUDA. This means:

- Local Ollama models run on CPU only - they are slower than GPU systems
- Keep local models at 7B parameters max (Phi-3, Mistral 7B, Hermes 7B, Qwen2.5 7B)
- All heavy reasoning, writing, and production tasks route to Claude or OpenRouter (cloud)
- Local models are for: fast lookups, offline fallback, embedding generation, cost reduction, and free high-volume tasks (Gemma3:12b via Ollama)

SECTION 1: BUILD ORIENTATION - YOUR AI OPERATING SYSTEM

1.1 What You Are Building

This manual is the exact blueprint Dr. D used to build his own AI-powered business operating system. You are using it to build yours. Every tool, every agent, every automation in this guide is battle-tested on a live production system generating real revenue.

By the end of this build, you will have:

- A 24/7 AI system running on a $30/month VPS that works while you sleep
- A content engine that produces TikTok scripts, LinkedIn posts, and email sequences automatically
- A personal AI staff of specialist agents handling research, revenue, content, and operations
- Real AI engineering skills on your resume - you built and deployed a production system
- A foundation you can monetize: sell the system, license the agents, or use it to run your own business

What you build	Your own AI operating system - 25-agent fleet on Hetzner VPS
Your role	Founder + Builder - this is your system, your business, your IP
Infrastructure	Hetzner VPS (~$8-30 CAD/mo depending on tier)
Time to first working agent	2-4 hours (Phase 0 complete)
Time to full 25-agent fleet	30-60 days at 4-6 hours/week
Revenue potential	Use your AI OS to run content, sales, or client services from day one

> **NOTE** *Dr. D built this system first. The manual documents every decision, every mistake, and every fix. You are not starting from scratch - you are starting from a proven blueprint.*

1.2 Two Types of Builders Using This Manual

Type A - Independent Builder (most readers)

You bought this manual to build your own AI OS for your own business or income stream. You work independently. You own everything you build. Sections 2 onward apply to you directly. Replace all references to Dr. D's business context with your own.

Type B - Co-Builder (1-2 students working directly with Dr. D)

A small number of students work directly with Dr. D on his production system as a HireEd Nexus Labs work-integrated learning placement. If this is you, Dr. D will confirm it explicitly. Section 1.3 and 1.4 apply to you specifically.

> **NOTE** *If you are unsure which type you are, you are Type A. Build your own system.*

1.3 Co-Builder Protocol (Type B Only - Skip if Independent)

Daily Standup (Required Every Work Day)

Send this message to Dr. D BEFORE starting work each day:

```
DATE: [Today's Date]
YESTERDAY: [What you completed]
TODAY: [What you plan to build or configure]
BLOCKERS: [Anything stopping you - be specific, not vague]
QUESTIONS: [Any decisions you need from Dr. D before proceeding]
```

> **NOTE** *Do not start working until you have sent the standup. This is non-negotiable.*

Escalation Rules

- Blocked for more than 30 minutes? Message Dr. D immediately
- Never spend hours stuck alone - time is money in this business

- If you break something: tell Dr. D instantly, explain what happened, state your rollback plan
- Never push untested code to dev or main
- Never share credentials via Discord, text, or email - always Bitwarden

Communication Channels

- Primary: Discord server (Dr. D owns the server - he sends you the invite link)
- Secondary: Email to andre@edvisingu.ca for formal updates
- Project Board: Notion (free) - Dr. D creates workspace and adds you as a guest
- Code: GitHub organization repository (Dr. D adds you as collaborator)
- Credentials: Bitwarden shared vault ONLY (Dr. D creates vault, invites you)

1.3 GitHub Workflow - Follow Exactly

main	Production-only. Dr. D approves ALL merges. You never push here directly.
dev	Active integration branch. All features merge here first.
feature/xxx	Your work branches (e.g., feature/hermes-agent, feature/content-pipeline)
fix/xxx	Bug fix branches

For every task:

1. Create a branch from dev: git checkout -b feature/task-name dev
2. Build and test locally before pushing
3. Commit with clear prefixes: feat: / fix: / config: / docs: / chore:
4. Example: git commit -m "feat: add Hermes agent persistent memory"
5. Push branch and open a Pull Request to dev
6. Tag Dr. D for review - never merge without his approval

1.4 Security Commitment

> **WARNING** You are handling real business credentials and intellectual property. Violations result in immediate removal.

- NEVER commit API keys, passwords, or secrets to GitHub
- ALWAYS add .gitignore before creating your first .env file
- ALWAYS use Bitwarden for all credential storage and sharing
- ALWAYS enable 2FA on every account you create for this project
- NEVER install software on Dr. D's machine without explicit written approval
- Keep a log of every account you create and every config change you make

SECTION 2: PRE-BUILD CHECKLIST

2.1 What You Need Before Starting (Student Side)

Task	Done?	Notes
Your own computer (Windows 10/11, macOS, or Ubuntu Linux)	[ ]	
Stable internet connection (minimum 25 Mbps download)	[ ]	
GitHub personal account created	[ ]	
VS Code installed on your machine	[ ]	
Discord account set up	[ ]	
Bitwarden account created (free tier is fine)	[ ]	
Tailscale installed on your machine (tailscale.com/download)	[ ]	
Read and understood Section 0 (Corrections) completely	[ ]	
Read and understood Section 1 (Communication Protocol) completely	[ ]	

2.2 Co-Builder Pre-Start Checklist (Type B - Skip if Independent)

If you are working directly with Dr. D on his production system, he must complete these steps before you touch anything:

Task	Done?	Notes
Bitwarden shared vault invitation sent to your email	[ ]	
GitHub repository created and you added as Collaborator	[ ]	

Discord server invite link sent	[ ]	
Notion workspace invite link (project board + SOPs + wiki)	[ ]	
Tailscale network invitation approved	[ ]	
Hetzner VPS provisioned and public IP shared via Bitwarden	[ ]	
Your SSH public key added to /home/deploy/.ssh/authorized_keys on VPS	[ ]	
Tailscale installed on VPS, your Tailscale invite sent	[ ]	
VPS deploy user credentials confirmed working	[ ]	
Confirmation of which accounts already exist (avoid duplicate creation)	[ ]	
Approved budget for any new subscriptions you need to create	[ ]	

2.2b Independent Builder - What YOU Set Up Yourself

If you are building your own system (Type A), you set everything up yourself. There is no client to wait on. Move directly to Section 3 after completing Section 2.1.

Task	Done?	Notes
Create your own Hetzner account at hetzner.com - choose CX22 (starter) or CPX41 (full build)	[ ]	
Create your own GitHub account and private repository	[ ]	
Create your own Bitwarden account (free tier) for secrets management	[ ]	
Create your own Discord server or use an existing communication channel	[ ]	

Create your own Anthropic account for Claude API access	[ ]	
Create your own OpenAI account for Codex/GPT access	[ ]	
Set up your own Google Workspace or Gmail account for automations	[ ]	
Sign up for n8n Cloud (free tier) or self-host n8n on your VPS	[ ]	

SECTION 3: REMOTE ACCESS SETUP

3.1 SSH to Hetzner VPS - Primary Method

Everything runs on the Hetzner VPS. Your local machine is just a terminal and code editor. You connect via SSH - no Windows Remote Desktop, no WSL2, no local installs required.

> **NOTE** *Complete Section 21 (Hetzner VPS Setup) first. Dr. D will provision the VPS and share the IP + deploy user credentials via Bitwarden before you start.*

Step 1 - Get SSH Access from Dr. D (He Provides This)

7. VPS public IP address stored in Bitwarden vault (format: 123.456.78.90)
8. SSH username: deploy (created during server hardening in Section 21)
9. Dr. D adds your public SSH key to /home/deploy/.ssh/authorized_keys on the VPS

Step 2 - Generate Your SSH Key (On Your Local Machine)

```
ssh-keygen -t ed25519 -C "your-email@example.com"   # Press Enter 3 times for defaults
cat ~/.ssh/id_ed25519.pub   # Copy the entire output and send to Dr. D
```

Step 3 - Connect to the VPS

```
ssh deploy@YOUR_HETZNER_IP   # Replace with actual IP from Bitwarden
```

You should see the Ubuntu 22.04 prompt. You are now inside the Hetzner VPS. All code runs here.

Step 4 - Add VS Code Remote SSH for IDE Access

10. Install VS Code on your local machine (https://code.visualstudio.com)
11. Open VS Code > Extensions > Search "Remote - SSH" - install it
12. Press F1 - type "Remote-SSH: Add New SSH Host" - enter: ssh deploy@YOUR_HETZNER_IP

13. Press F1 - type "Remote-SSH: Connect to Host" - select the VPS
14. VS Code now runs against the VPS filesystem. Open /opt/edvisingu as your workspace.

> **NOTE** *No WSL2, no Docker Desktop on Windows, no local Python installs needed. Your laptop just needs VS Code + SSH. The VPS is your Linux dev machine.*

3.2 Tailscale - Optional Private Mesh (Recommended for Teams)

Tailscale creates a private mesh between the VPS, Dr. D, and all student devs. Optional but useful for hiding the VPS admin ports from the public internet.

15. Install Tailscale on the VPS: curl -fsSL https://tailscale.com/install.sh | sh && sudo tailscale up
16. Install Tailscale on your laptop from https://tailscale.com/download
17. Dr. D invites your Tailscale email from the admin console
18. Once connected, you can also SSH via the Tailscale IP (100.x.x.x) instead of the public IP

3.3 Remote Access Rules

- Always message Dr. D on Telegram before starting a work session
- Only work during agreed hours unless Dr. D explicitly approves otherwise
- Never store secrets in project files - .env only, never committed
- Never touch other student dev workspaces in /home/ on the VPS
- Log off properly - do not leave idle SSH sessions open

SECTION 4: ACCOUNT SETUP

Confirm with Dr. D which accounts already exist before creating new ones. Store every API key in Bitwarden the moment you generate it.

4.1 AI Model Provider Accounts

Task	Done?	Notes
Anthropic (Claude) - https://console.anthropic.com - Add billing, generate API key, store in Bitwarden	[ ]	
OpenRouter - https://openrouter.ai - Create account, add credits ($20 to start), generate API key	[ ]	
OpenAI - https://platform.openai.com - Create account, add billing, generate API key (for Codex/GPT access)	[ ]	
Google AI Studio - https://aistudio.google.com - Connect to Dr. D's Google Workspace account, get Gemini API key	[ ]	

4.2 Infrastructure Accounts

Task	Done?	Notes
GitHub - https://github.com - Dr. D creates organization, adds you as collaborator with write access	[ ]	
Supabase - https://supabase.com - Create new project named "edvisingu-prod", region: Canada East (or closest)	[ ]	
Vercel - https://vercel.com - Connect to GitHub organization; free Hobby plan to start	[ ]	
Cloudflare - https://cloudflare.com - Add Dr. D's domains; use free plan for DNS + Pages	[ ]	

Railway - https://railway.app - For hosting persistent services (n8n in production, FastAPI)	[ ]	

4.3 Automation Accounts

Task	Done?	Notes
n8n Cloud - https://n8n.io - Start with cloud for easier setup; consider self-hosted later via Docker	[ ]	
Zapier - https://zapier.com - Confirm if Dr. D already has this before creating new account	[ ]	
Make.com - https://make.com - Optional; only use if n8n cannot handle a specific integration	[ ]	

4.4 Content Creation Accounts

Task	Done?	Notes
HeyGen - https://heygen.com - AI avatar video generation; confirm if Dr. D already has account + avatar	[ ]	
ElevenLabs - https://elevenlabs.io - AI voice cloning; requires Dr. D's voice sample for cloning	[ ]	
Canva - https://canva.com - Confirm Dr. D has Pro or Teams; request access to brand kit	[ ]	
Riverside.fm - https://riverside.fm - AI podcast and video recording, editing, and transcription	[ ]	
Higgsfield - https://higgsfield.ai - AI cinematic video for social content	[ ]	
Blotato - https://blotato.com - AI-native social media scheduling, repurposing, and multi-platform publishing	[ ]	

4.5 Business Platform Accounts

Task	Done?	Notes
Whop - https://whop.com - Dr. D owns this; request webhook credentials from his admin panel	[ ]	
Discord - Dr. D owns the community server; he sends you an invite with appropriate role (developer access)	[ ]	
Google Workspace - Confirm Dr. D is admin; request access to the project Shared Drive folder	[ ]	
Notion - https://notion.so - Free plan; Dr. D creates workspace and adds you as guest - used for project board, SOPs, wiki, and documentation	[ ]	
Bitwarden (free) - https://bitwarden.com - Free tier; Dr. D creates a free Organization (up to 2 users at no cost), invites you to the shared vault for secure credential sharing	[ ]	

WARNING Store ALL credentials in Bitwarden the moment you create them. No exceptions. Not in a text file, not in Discord, not in email.

SECTION 5: PHASE 1 - DEVELOPMENT ENVIRONMENT SETUP

The Hetzner VPS is the build and runtime environment. All code, services, and Docker containers run there. Your local machine only needs VS Code with Remote SSH. Complete Section 21 (Hetzner VPS Setup) before starting this section.

5.1 Local Machine Setup - SSH + VS Code Only

You do NOT install WSL2, Docker, or Python locally. The Hetzner VPS is already running Ubuntu 22.04. Your local machine just needs SSH access and VS Code.

Step 1 - Install VS Code on your machine (https://code.visualstudio.com)

Step 2 - Install the Remote SSH extension inside VS Code:

19. Open VS Code > Extensions (Ctrl+Shift+X) > Search "Remote - SSH" > Install
20. Also install: Python, GitLens, DotENV, Docker, Thunder Client

Step 3 - Connect to Hetzner via Remote SSH (see Section 3 for SSH key setup):

```
# Press F1 in VS Code and type:
Remote-SSH: Connect to Host
# Enter: deploy@YOUR_HETZNER_IP
```

Step 4 - Once connected, open the project folder on the VPS:

```
/opt/edvisingu
```

> **NOTE** *VS Code is now running against the VPS filesystem. Files you edit, terminals you open, and commands you run are all on Hetzner. Your laptop is just the display.*

5.2 Node.js v20 - Run These on the VPS via SSH

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

```
sudo apt install -y nodejs
node --version   # Confirm: v20.x.x
npm --version    # Confirm: 10.x.x
sudo npm install -g pm2   # Process manager  -  keeps services running after
disconnects
```

5.3 Docker - Already Installed on the VPS

Docker runs natively on Hetzner Ubuntu 22.04. It was installed during Section 21 server setup. You do NOT install Docker Desktop on your local machine.

Verify Docker is running on the VPS (run via SSH):

```
docker --version         # Should show Docker 24+
docker compose version   # Should show v2.x
docker ps                # Lists running containers
```

> **NOTE** *All docker compose commands run on the VPS. Your local machine never needs Docker.*

5.4 Git + SSH Key for GitHub - Run on the VPS via SSH

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
ssh-keygen -t ed25519 -C "your-email@example.com"   # Press Enter 3 times for
defaults
cat ~/.ssh/id_ed25519.pub   # Copy the entire output
```

Go to GitHub > Settings > SSH and GPG Keys > New SSH Key > Paste the key > Save

Clone the project repo:

```
mkdir -p /opt/edvisingu && cd /opt/edvisingu
git clone git@github.com:edvisingu/ai-ecosystem.git
cd ai-ecosystem && git checkout dev
```

5.5 VS Code Extensions - Install on Your Local Machine

VS Code runs locally but connects to the VPS via Remote SSH (set up in Section 5.1). Install these extensions:

- Remote - SSH (connects VS Code directly to Hetzner VPS - REQUIRED)
- Python + Pylance
- Docker
- GitLens
- Tailwind CSS IntelliSense
- DotENV (highlights .env files)
- Prettier - Code formatter
- Thunder Client (test APIs inside VS Code - no Postman needed)
- ES7+ React/Redux/React-Native snippets

Open the project on the VPS via Remote SSH:

```
# In VS Code: F1 > Remote-SSH: Connect to Host > deploy@YOUR_HETZNER_IP
# Then: File > Open Folder > /opt/edvisingu
```

5.6 Python Virtual Environment - Run on VPS via SSH

The Python environment lives on the Hetzner VPS. Run these commands inside your SSH session:

```
cd /opt/edvisingu
python3.11 -m venv venv
source venv/bin/activate   # You should see (venv) in your terminal
pip install --upgrade pip
```

Install all required Python packages:

```
pip install fastapi uvicorn langchain langchain-community langchain-anthropic
pip install langchain-openai chromadb supabase python-dotenv openai anthropic
pip install sentence-transformers tiktoken httpx pydantic schedule pdfplumber
```

Save dependencies:

```
pip freeze > requirements.txt
git add requirements.txt && git commit -m "chore: add Python requirements"
```

5.7 Folder Structure - Create on VPS via SSH

Run these on the Hetzner VPS. The project root is /opt/edvisingu.

```
sudo mkdir -p /opt/edvisingu/{agents,memory,content,automation,research}
sudo mkdir -p /opt/edvisingu/{courses,vector-db,scripts,dashboards,prompts}
sudo mkdir -p /opt/edvisingu/{workflows,brand-assets,api-configs,deployments}
sudo mkdir -p /opt/edvisingu/agents/{hermes,content-agent,student-advisor,career-coach,credihire,social-agent}
sudo mkdir -p /opt/edvisingu/vector-db/chroma-data
sudo chown -R deploy:deploy /opt/edvisingu
```

5.8 Environment Variables - On Hetzner VPS Only

> **WARNING** Create .gitignore BEFORE you create any .env file. This order matters.

Step 1 - Create .gitignore:

```
cat > /opt/edvisingu/.gitignore << 'EOF'
.env
.env.local
.env.*
venv/
__pycache__/
*.pyc
.DS_Store
node_modules/
chroma-data/
*.log
EOF
git add .gitignore && git commit -m "chore: add gitignore"
```

Step 2 - Create the .env file (fill values from Bitwarden):

```
# ── AI PROVIDERS ──────────────────────────────────
ANTHROPIC_API_KEY=sk-ant-api03-...
OPENAI_API_KEY=sk-proj-...
OPENROUTER_API_KEY=sk-or-v1-...
GOOGLE_AI_API_KEY=AIzaSy...
```

```
# ── SUPABASE ──────────────────────────────────────────
SUPABASE_URL=https://xxxxxxxxxxxxx.supabase.co
SUPABASE_ANON_KEY=eyJhbGci...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGci...

# ── N8N ───────────────────────────────────────────────
N8N_BASIC_AUTH_USER=admin
N8N_BASIC_AUTH_PASSWORD=ChangeThisStrongPassword123!
N8N_WEBHOOK_URL=http://localhost:5678

# ── CONTENT TOOLS ─────────────────────────────────────
HEYGEN_API_KEY=...
ELEVENLABS_API_KEY=...

# ── APP ───────────────────────────────────────────────
ENVIRONMENT=development
SECRET_KEY=paste-64-char-random-string-here
```

Generate a secure SECRET_KEY:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

> **NOTE** *.env is already in .gitignore. Verify with: git status (it should NOT appear as untracked)*

Phase 1 Completion Checklist

Task	Done?	Notes
Hetzner VPS provisioned (Section 21 complete)	[ ]	
SSH key generated locally, public key sent to Dr. D, added to VPS	[ ]	
SSH connection to deploy@YOUR_HETZNER_IP confirmed working	[ ]	
VS Code Remote SSH extension installed and connecting to VPS	[ ]	

VS Code opens /opt/edvisingu on the VPS successfully	[ ]	
Docker running on VPS: docker --version + docker ps return results	[ ]	
Node.js v20+ confirmed on VPS (node --version)	[ ]	
Python 3.11 venv created at /opt/edvisingu/venv, all packages installed	[ ]	
Git configured on VPS with SSH key added to GitHub	[ ]	
Project repo cloned to /opt/edvisingu on dev branch	[ ]	
All directories in /opt/edvisingu folder structure created	[ ]	
.gitignore committed BEFORE any .env file	[ ]	
.env file created on VPS with all placeholders filled (NOT committed)	[ ]	
Verify: git status shows .env as ignored (not untracked)	[ ]	
All API keys stored in Bitwarden vault	[ ]	
Phase 1 standup sent to Dr. D confirming completion	[ ]	

SECTION 6: PHASE 2 - VPS AI STACK

6.1 Ollama - Local Model Runner

Run on the Hetzner VPS via SSH:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama --version   # Confirm install
```

Start the Ollama service:

```
ollama serve &   # Runs in background on port 11434
```

Pull required models (large downloads - do this on a good connection):

```
ollama pull phi3              # 2.3 GB  -  Fast, good for quick classification tasks
ollama pull mistral           # 4.1 GB  -  Balanced performance and speed
ollama pull gemma3:12b        # 8.1 GB  -  Free Gemma 3 (12B params) for zero-cost
local tasks
ollama pull qwen2.5:7b        # 4.4 GB  -  Strong reasoning and code
ollama pull nomic-embed-text  # 274 MB  -  REQUIRED for vector embeddings and memory
```

For the Hermes model (Dr. D's AI persona):

```
ollama pull nous-hermes2   # The Hermes 2 model by NousResearch
```

Test Ollama is working:

```
ollama run phi3 "Introduce yourself as part of the EdVisingU AI system"
```

Make Ollama auto-start when the VPS boots (add to /home/deploy/.bashrc):

```
echo "pgrep ollama > /dev/null || ollama serve > /tmp/ollama.log 2>&1 &" >> ~/.bashrc
```

List all downloaded models:

```
ollama list
```

6.2 ChromaDB - Vector Memory Database

ChromaDB stores AI memory as vector embeddings for semantic search and retrieval (RAG).

ChromaDB was installed as part of the pip requirements. Test it now:

```
cd /opt/edvisingu/vector-db
cat > test_chroma.py << 'EOF'
import chromadb
from chromadb.config import Settings
client = chromadb.PersistentClient(path="./chroma-data")
col = client.get_or_create_collection("test")
col.add(documents=["EdVisingU teaches students to earn while learning",
                   "HireEd Nexus Labs connects students to real projects",
                   "CrediVersity offers AI micro-credentials"],
        ids=["1","2","3"])
result = col.query(query_texts=["student income opportunities"], n_results=2)
print("ChromaDB working! Top results:", result["documents"])
EOF
source ../venv/bin/activate && python3 test_chroma.py
```

Expected output: ChromaDB working! Top results: with relevant documents returned.

6.3 n8n - Workflow Automation Backbone

n8n is the orchestration system that connects all services. Run it via Docker Compose.

Create the Docker Compose file:

```
cat > /opt/edvisingu/automation/docker-compose.yml << 'EOF'
version: "3.8"
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n:latest
    restart: always
    ports:
      - "5678:5678"
    environment:
      - N8N_BASIC_AUTH_ACTIVE=true
      - N8N_BASIC_AUTH_USER=${N8N_BASIC_AUTH_USER}
      - N8N_BASIC_AUTH_PASSWORD=${N8N_BASIC_AUTH_PASSWORD}
      - N8N_HOST=localhost
      - N8N_PORT=5678
```

```
      - WEBHOOK_URL=http://localhost:5678
      - GENERIC_TIMEZONE=America/Toronto
    volumes:
      - n8n_data:/home/node/.n8n
      - ./workflows:/workflows
volumes:
  n8n_data:
EOF
```

Start n8n:

```
cd /opt/edvisingu/automation
docker compose --env-file ../.env up -d
```

Verify n8n is running:

```
docker ps | grep n8n   # Should show running container
```

Access n8n in browser at: http://localhost:5678

Login with N8N_BASIC_AUTH_USER and N8N_BASIC_AUTH_PASSWORD from .env

> **NOTE** *For production, n8n should deploy to Railway. For development, localhost is fine.*

6.4 FastAPI - AI Orchestration Service

This is the specialist agent router. It runs at port 8000 on Hetzner and receives tasks delegated by OpenJarvis or directly from the hermes-agent dispatch skill. It routes each request to the correct specialist agent container.

Create the main application file:

```
cat > /opt/edvisingu/agents/main.py << 'EOF'
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from dotenv import load_dotenv
import os, anthropic
```

```
load_dotenv("../.env")
app = FastAPI(title="EdVisingU AI Orchestration API", version="1.0.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

class ChatRequest(BaseModel):
    message: str
    model: str = "claude"
    agent: str = "hermes"
    max_tokens: int = 2048

class ContentRequest(BaseModel):
    topic: str
    platforms: list[str] = ["linkedin", "tiktok", "email"]
    tone: str = "bold, direct, and insightful"

@app.get("/health")
def health(): return {"status": "running", "version": "1.0.0", "ecosystem":
"EdVisingU"}

@app.post("/chat")
async def chat(req: ChatRequest):
    client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
    msg = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=req.max_tokens,
        messages=[{"role": "user", "content": req.message}]
    )
    return {"response": msg.content[0].text, "model": "claude", "agent": req.agent}

@app.post("/content/generate")
async def generate_content(req: ContentRequest):
    client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
    prompt = f"""You are Dr. Andre De Freitas, founder of EdVisingU.
Write content about: {req.topic}
Tone: {req.tone}
Generate for platforms: {", ".join(req.platforms)}
Return JSON with platform names as keys and content as values."""
    msg = client.messages.create(model="claude-sonnet-4-6", max_tokens=4096,
        messages=[{"role": "user", "content": prompt}])
    return {"topic": req.topic, "content": msg.content[0].text}
```

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000, reload=True)
EOF
```

Run the API:

```
cd /opt/edvisingu/agents
source ../venv/bin/activate
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Test the health endpoint (in a new terminal):

```
curl http://localhost:8000/health
```

Expected: {"status":"running","version":"1.0.0","ecosystem":"EdVisingU"}

Test the Claude chat endpoint:

```
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "What is EdVisingU and how does it help students?"}'
```

Phase 2 Completion Checklist

Task	Done?	Notes
Ollama installed on Hetzner VPS and responding to "ollama --version"	[ ]	
phi3 model pulled and generating responses via "ollama run phi3"	[ ]	
mistral model pulled and generating responses	[ ]	
nous-hermes2 model pulled (Hermes persona model)	[ ]	
nomic-embed-text model pulled (required for memory system)	[ ]	
ChromaDB test script runs successfully and returns relevant results	[ ]	

n8n running via Docker Compose on port 5678	[ ]	
n8n dashboard accessible and login working	[ ]	
FastAPI running on port 8000 with "reload" mode active	[ ]	
Health check returns 200 OK with correct JSON	[ ]	
Claude chat endpoint returns AI response via FastAPI	[ ]	
All three services running simultaneously (Ollama, n8n, FastAPI)	[ ]	
Phase 2 standup sent to Dr. D confirming completion	[ ]	

SECTION 7: PHASE 3 - HERMES AGENT BUILD

Hermes is Dr. D's persistent AI executive assistant. It remembers conversations, retrieves relevant knowledge, and acts as the primary AI brain of the ecosystem.

7.1 Hermes System Prompt

```
cat > /opt/edvisingu/prompts/hermes_system.txt << 'EOF'
You are Hermes, the AI Executive Assistant for Dr. Andre De Freitas (Dr. D).
Dr. D is a senior higher education leader, AI strategist, and founder of:
EdVisingU, CrediVersity, DrD Learn, and HireEd Nexus Labs.

Your responsibilities:
- Execute tasks, research requests, and content workflows
- Retrieve and apply persistent memory from previous sessions
- Coordinate between specialized agents (content, research, advisor, sales)
- Think like a strategic operator and monetization expert
- Be direct, specific, and execution-focused  -  never vague
- Always tie outputs to revenue, efficiency, or growth

Key context:
- Primary goal: Build a $1M+ AI-powered education ecosystem
- Revenue streams: memberships, courses, coaching, AI services, SaaS
- Platforms: edvisingu.com, crediversity.com, ai.edvisingu.com, gohireed.com
- Tools: n8n, Supabase, Vercel, Cloudflare, Whop, Discord
EOF
```

7.2 Hermes Agent with Persistent Memory

```
cat > /opt/edvisingu/agents/hermes/agent.py << 'EOF'
from langchain_anthropic import ChatAnthropic
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import OllamaEmbeddings
from langchain.memory import ConversationBufferWindowMemory
from langchain.chains import ConversationalRetrievalChain
from langchain.schema import SystemMessage
from dotenv import load_dotenv
import os
```

```
load_dotenv("../../.env")

# Load system prompt
with open("../../prompts/hermes_system.txt") as f:
    SYSTEM_PROMPT = f.read()

# Embeddings via local Ollama
embeddings = OllamaEmbeddings(model="nomic-embed-text")

# Persistent ChromaDB memory
vectorstore = Chroma(
    persist_directory="../../vector-db/chroma-data",
    embedding_function=embeddings,
    collection_name="hermes_memory"
)

# Claude as the reasoning LLM
llm = ChatAnthropic(
    model="claude-sonnet-4-6",
    api_key=os.getenv("ANTHROPIC_API_KEY"),
    max_tokens=4096
)

# Conversation memory (last 10 exchanges)
memory = ConversationBufferWindowMemory(
    k=10, memory_key="chat_history",
    return_messages=True, output_key="answer"
)

# Build retrieval chain
chain = ConversationalRetrievalChain.from_llm(
    llm=llm,
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
    memory=memory,
    return_source_documents=False,
    verbose=False
)

def chat(message: str) -> str:
    full_message = f"{SYSTEM_PROMPT}\n\nUser: {message}"
    result = chain({"question": full_message})
```

```
    return result["answer"]


if __name__ == "__main__":
    print("Hermes Agent ready. Type quit to exit.")
    while True:
        user_input = input("Dr. D: ")
        if user_input.lower() in ["quit", "exit"]: break
        print(f"Hermes: {chat(user_input)}")
EOF
```

7.3 Seed Hermes Knowledge Base

Give Hermes foundational knowledge about the ecosystem before first use:

```
cat > /opt/edvisingu/agents/hermes/seed_knowledge.py << 'EOF'
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import OllamaEmbeddings
import sys; sys.path.append("../..")

knowledge = [
    "EdVisingU is an AI-powered education platform at edvisingu.com that teaches students how to earn income while studying.",
    "CrediVersity at crediversity.com offers AI micro-credentials and digital learning pathways for career transformation.",
    "DrD Learn is the course and digital product platform for Dr. Andre De Freitas.",
    "HireEd Nexus Labs at gohireed.com connects students to real work-integrated learning projects and income pathways.",
    "ai.edvisingu.com is the AI assistant hub for the EdVisingU ecosystem.",
    "Primary revenue streams: $47/month memberships on Whop, course sales on Skillplate.com and Gumroad, Etsy digital products, high-ticket coaching at $999+, AI consulting services, and SaaS products.",
    "Monetization platforms: Skillplate.com (owned storefront), Whop (memberships + community), Gumroad (digital downloads), Etsy (marketplace - semi-manual), Discord (community + Whop-gated access), Vercel (web apps).",
    "Content channels: TikTok, LinkedIn, YouTube Shorts, Instagram Reels, email newsletter.",
    "Tech stack: Next.js, Supabase, Vercel, n8n, Cloudflare, Ollama, Claude API, OpenRouter, FastAPI.",
    "OSAP and BSWD are Canadian student funding programs that are part of the ecosystem's automation focus.",
    "The student developer build the AI infrastructure remotely - Dr. D is the founder and strategic lead.",
]
```

```
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma(persist_directory="../../vector-db/chroma-data",
                     embedding_function=embeddings, collection_name="hermes_memory")
ids = [f"seed_{i}" for i in range(len(knowledge))]
vectorstore.add_texts(texts=knowledge, ids=ids)
print(f"Knowledge seeded: {len(knowledge)} documents added to Hermes memory.")
EOF
```

Run the seeder:

```
cd /opt/edvisingu/agents/hermes
source ../../venv/bin/activate
python3 seed_knowledge.py
```

Test Hermes in terminal:

```
python3 agent.py
# Type: "What platforms does Dr. D use for monetization?"
# Hermes should retrieve the correct information from memory
```

7.4 Expose Hermes via FastAPI

Add this endpoint to /opt/edvisingu/agents/main.py:

```
from hermes.agent import chat as hermes_chat


@app.post("/hermes/chat")
async def hermes_endpoint(req: ChatRequest):
    response = hermes_chat(req.message)
    return {"response": response, "agent": "hermes"}
```

SECTION 8: PHASE 3 - SUPABASE DATABASE SETUP

8.1 Connect to Existing Supabase Project - DO NOT CREATE A NEW ONE

> **WARNING** CRITICAL: Dr. D already has a live Supabase project (ID: bvxhicdnaordolguuyal). Do NOT create a new project. Creating a new one will destroy the existing schema and data. Ask Dr. D for credentials via Bitwarden before touching anything.

Steps to connect to the existing project:

21. Go to https://supabase.com and sign in with Dr. D's Google Workspace account
22. Select project: bvxhicdnaordolguuyal from the project dashboard
23. Go to Settings > API to retrieve the Project URL and service_role key
24. Store both in Bitwarden and update SUPABASE_URL and SUPABASE_KEY in the VPS master .env
25. Go to Database > Tables to review what already exists before running any migrations
26. Only run new migrations if Dr. D confirms the table does not already exist

8.2 Enable pgvector and Run Schema

In Supabase dashboard, go to Database > SQL Editor. Run this script (save as api-configs/schema.sql):

```
-- Enable vector extension
CREATE EXTENSION IF NOT EXISTS vector;


-- AI Memory table (for Supabase-based vector search as alternative to ChromaDB)
CREATE TABLE IF NOT EXISTS ai_memory (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  agent_name TEXT NOT NULL,
  content TEXT NOT NULL,
  embedding VECTOR(1536),
  metadata JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

```
-- Leads / Student CRM
CREATE TABLE IF NOT EXISTS leads (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  name TEXT,
  email TEXT UNIQUE,
  source TEXT,
  status TEXT DEFAULT 'new',
  tags TEXT[],
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Content Pipeline Queue
CREATE TABLE IF NOT EXISTS content_queue (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  topic TEXT NOT NULL,
  platform TEXT NOT NULL,
  status TEXT DEFAULT 'pending',
  raw_content TEXT,
  final_content TEXT,
  scheduled_at TIMESTAMPTZ,
  published_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Products / Courses / Offers
CREATE TABLE IF NOT EXISTS products (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  name TEXT NOT NULL,
  type TEXT NOT NULL,
  price NUMERIC,
  description TEXT,
  platform TEXT,
  url TEXT,
  active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Members / Subscribers
CREATE TABLE IF NOT EXISTS members (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
```

```
  email TEXT UNIQUE NOT NULL,
  name TEXT,
  whop_id TEXT,
  plan TEXT,
  status TEXT DEFAULT 'active',
  joined_at TIMESTAMPTZ DEFAULT now()
);

-- Enable Row Level Security on all tables
ALTER TABLE ai_memory ENABLE ROW LEVEL SECURITY;
ALTER TABLE leads ENABLE ROW LEVEL SECURITY;
ALTER TABLE content_queue ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE members ENABLE ROW LEVEL SECURITY;
```

NOTE *RLS is ON but no policies are set yet - meaning only service_role key can access the tables. We add user-level policies when auth is implemented.*

8.3 Storage Buckets

In Supabase dashboard > Storage, create these buckets:

Task	Done?	Notes
brand-assets (Public: YES) - Logos, brand images, Canva exports, social media assets	[ ]	
course-content (Public: NO) - Course videos, PDFs, lesson materials	[ ]	
resumes (Public: NO) - CrediHire resume uploads from users	[ ]	
avatars (Public: YES) - HeyGen and AI avatar output files	[ ]	
audio (Public: YES) - ElevenLabs voice output files	[ ]	

SECTION 9: PHASE 4 - CONTENT AUTOMATION SYSTEM

9.1 Content Pipeline Architecture

Input	Topic string (manual input, scheduled trigger, or RSS trending feed)
Step 1	Claude generates long-form content (1500-2000 words)
Step 2	Content split into: LinkedIn post, TikTok script, YouTube Short, email newsletter
Step 3	Optional: Dr. D reviews and approves (or auto-publishes on schedule)
Step 4	HeyGen generates avatar video from TikTok script
Step 5	ElevenLabs generates voice-over audio
Step 6	Blotato schedules and repurposes posts across all platforms
Step 7	Supabase content_queue table updated with status "published"

9.2 n8n Content Factory Workflow

In n8n at http://localhost:5678, create a new workflow named "Content Factory". Build these nodes in order:

27. Webhook node - Method: POST, Path: /content - This is the workflow trigger
28. HTTP Request node - POST to http://localhost:8000/content/generate with body: {"topic": "{{$json.topic}}"}
29. Code node - Parse the JSON response and extract platform-specific content
30. Supabase node - Insert each platform's content into the content_queue table
31. IF node - Check if "auto_publish" flag is true or false
32. HTTP Request node (Blotato) - POST to Blotato API to schedule and repurpose posts across all platforms

33. Email node - Send newsletter content to email provider (Gmail or ConvertKit)

Export the workflow:

In n8n, go to workflow menu > Download. Save as /opt/edvisingu/workflows/content-factory.json

```
git add workflows/ && git commit -m "feat: add content factory n8n workflow"
```

9.3 Platform Content Format Rules

Code these as part of the content generation prompt in FastAPI:

- LinkedIn: 1200-1500 characters. Strong hook in line 1. No hashtag spam (max 3). Professional but human tone.
- TikTok Script: 60-90 seconds spoken. First 3 seconds = bold hook. End with strong CTA. Use short punchy sentences.
- YouTube Short: Same structure as TikTok but slightly longer - up to 60 seconds on screen.
- Email Newsletter: Subject line + preview text + body. Value-first, no fluff. One clear CTA at the end.
- Carousel (LinkedIn/Instagram): 5-7 slides. Slide 1 = hook. Slide 2-6 = value points. Slide 7 = CTA.

SECTION 10: PHASE 4 - SPECIALIZED AGENT BUILDS

10.1 Student AI Advisor Agent

Helps students find courses, plan careers, identify income opportunities, and connects them to HireEd Nexus Labs.

Create /opt/edvisingu/agents/student-advisor/advisor.py:

```
ADVISOR_SYSTEM = """
You are the EdVisingU Student AI Advisor.
Your mission: help students succeed academically, earn income while studying, and build career-ready skills.
Always recommend specific EdVisingU programs, HireEd Nexus Labs opportunities, or CrediVersity credentials.
Be encouraging, action-oriented, and specific. Never give vague advice.
"""
```

Add to main.py:

```
@app.post("/advisor/chat")
async def advisor_chat(req: ChatRequest):
    client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
    msg = client.messages.create(model="claude-sonnet-4-6", max_tokens=2048,
        system=ADVISOR_SYSTEM,
        messages=[{"role": "user", "content": req.message}])
    return {"response": msg.content[0].text, "agent": "student-advisor"}
```

10.2 CrediHire Resume Agent

Analyzes resumes, scores ATS compatibility, and generates optimized versions.

Install PDF parsing:

```
pip install pdfplumber
```

Key endpoints to build in main.py:

- POST /credihire/analyze - Accept resume text, return ATS score + specific improvement feedback

- POST /credihire/optimize - Rewrite resume sections targeting a specific job description
- POST /credihire/linkedin-summary - Generate optimized LinkedIn About section from resume
- POST /credihire/cover-letter - Generate targeted cover letter from resume + job posting

Example analyze endpoint:

```
@app.post("/credihire/analyze")
async def analyze_resume(resume_text: str):
    client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
    prompt = f"""Analyze this resume for ATS compatibility and overall strength.
Return JSON with: ats_score (0-100), strengths (list), weaknesses (list),
improvements (list).
Resume:\n{resume_text}"""
    msg = client.messages.create(model="claude-sonnet-4-6", max_tokens=2048,
        messages=[{"role": "user", "content": prompt}])
    return {"analysis": msg.content[0].text}
```

10.3 Social Media Trend Agent (Automated)

Build as an n8n workflow that runs daily:

34. Schedule trigger: every day at 8:00 AM EST
35. HTTP node: fetch trending topics from Google Trends API or RSS feeds for AI + education
36. HTTP node: POST top 3 topics to /content/generate endpoint
37. Supabase node: insert generated content with status "pending_review"
38. Gmail node: email Dr. D a digest of today's generated content for approval

SECTION 11: PHASE 4 - NEXT.JS DASHBOARD

11.1 Project Setup

```
cd /opt/edvisingu/dashboards
npx create-next-app@latest edvisingu-dashboard --typescript --tailwind --eslint --app
--src-dir
cd edvisingu-dashboard
npm install @supabase/supabase-js @supabase/auth-helpers-nextjs
npm install @headlessui/react lucide-react recharts
```

Create .env.local (this stays local, not committed):

```
NEXT_PUBLIC_SUPABASE_URL=your-supabase-url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-supabase-anon-key
NEXT_PUBLIC_API_URL=http://localhost:8000
```

Test the dev server:

```
npm run dev   # Accessible at http://localhost:3000
```

11.2 Dashboard Pages to Build

Task	Done?	Notes
/dashboard - Overview: lead count, content queue status, recent Hermes activity, revenue snapshot	[ ]	
/dashboard/hermes - Chat interface with Hermes agent (POST to /hermes/chat API)	[ ]	
/dashboard/content - Content factory: input topic, view generated content, approve or schedule	[ ]	
/dashboard/leads - CRM table view of all leads from Supabase leads table	[ ]	
/dashboard/advisor - Student Advisor chat interface (embeddable on public site)	[ ]	

/dashboard/credihire - Resume upload, analysis results display, download optimized version	[ ]	
/dashboard/analytics - Revenue by stream, content performance, member growth	[ ]	

11.3 Deploy to Vercel

39. Push Next.js project to GitHub repo (in the dashboards/ subdirectory or as a separate repo)
40. Go to https://vercel.com > Import Project > Select the GitHub repo
41. Set Root Directory to: dashboards/edvisingu-dashboard (or root if separate repo)
42. Add environment variables in Vercel dashboard (same as .env.local)
43. Deploy - Vercel auto-deploys on every push to main
44. In Vercel dashboard > Domains > Add custom domain (e.g., app.edvisingu.com)
45. In Cloudflare DNS for edvisingu.com, add CNAME record pointing to Vercel's URL

SECTION 12: PLATFORM INTEGRATIONS

12.1 HeyGen - AI Avatar Videos

Generates video of Dr. D's AI avatar speaking from a script. Needs his avatar_id from HeyGen dashboard.

Add to agents/main.py:

```
import httpx


@app.post("/heygen/generate")
async def heygen_video(script: str, avatar_id: str):
    url = "https://api.heygen.com/v2/video/generate"
    headers = {"X-Api-Key": os.getenv("HEYGEN_API_KEY"), "Content-Type":
"application/json"}
    payload = {
        "video_inputs": [{
            "character": {"type": "avatar", "avatar_id": avatar_id},
            "voice": {"type": "text", "input_text": script, "speed": 1.0}
        }],
        "dimension": {"width": 1080, "height": 1920},
        "aspect_ratio": "9:16"
    }
    async with httpx.AsyncClient() as client:
        r = await client.post(url, json=payload, headers=headers)
        return r.json()
```

> **NOTE** *Get Dr. D's avatar_id from his HeyGen account at app.heygen.com > Avatars. Store it in Bitwarden.*

12.2 ElevenLabs - AI Voice

```
pip install elevenlabs
```

```
@app.post("/elevenlabs/speak")
async def text_to_speech(text: str, voice_id: str):
    from elevenlabs.client import ElevenLabs
```

```
    client = ElevenLabs(api_key=os.getenv("ELEVENLABS_API_KEY"))
    audio = client.generate(text=text, voice=voice_id, model="eleven_monolingual_v1")
    path = f"/tmp/audio_{hash(text)}.mp3"
    with open(path, "wb") as f:
        for chunk in audio: f.write(chunk)
    return {"audio_path": path, "voice_id": voice_id}
```

> **NOTE** *Voice ID is in Dr. D's ElevenLabs dashboard under Voices. Requires his voice to be cloned first - this is a manual step Dr. D does himself.*

12.3 Google Workspace via n8n

46. In n8n > Settings > Credentials > Add Credential > Google OAuth2 API
47. Create OAuth2 credentials in Google Cloud Console (console.cloud.google.com)
48. Enable APIs: Gmail API, Google Drive API, Google Calendar API, Google Sheets API
49. Set redirect URI to your n8n instance URL
50. Test each Google service with a simple n8n workflow before building full automations

12.4 Whop Webhook Integration

51. In Whop dashboard > Developer > Webhooks > Create Webhook
52. Webhook URL: http://[your-n8n-url]/webhook/whop-events
53. Subscribe to: member.created, member.deleted, payment.completed, payment.failed
54. In n8n, create a Webhook trigger workflow named "Whop Events"
55. Add Supabase node: INSERT into members table on member.created
56. Add Gmail node: send welcome email to new member
57. Add Supabase node: UPDATE member status on member.deleted

12.5 Discord Integration

Discord is the primary community platform. Build automation that connects Whop membership events to Discord roles and sends AI-generated announcements.

Discord Bot Setup

58. Go to https://discord.com/developers/applications and create a New Application
59. Name it: EdVisingU Bot
60. Go to Bot tab > Add Bot > Copy the Bot Token - store in Bitwarden
61. Go to OAuth2 > URL Generator - select bot scope + Send Messages, Manage Roles, Read Messages permissions
62. Use the generated URL to invite the bot to Dr. D's Discord server

n8n Discord Workflows to Build

63. Whop member.created event triggers n8n > bot assigns "Member" role in Discord > bot sends welcome DM
64. New content published in content_queue triggers bot announcement in #announcements channel
65. Daily standup automation: bot posts daily prompt in #student-dev channel at 9 AM EST
66. New HireEd opportunity posted > bot notifies #opportunities channel with formatted embed

Add Discord Bot Token to .env:

```
DISCORD_BOT_TOKEN=your-bot-token-from-bitwarden
DISCORD_SERVER_ID=your-server-id
DISCORD_MEMBER_ROLE_ID=role-id-for-paid-members
```

> **NOTE** *Get Server ID and Role IDs by enabling Developer Mode in Discord: User Settings > Advanced > Developer Mode, then right-click any server/role and click Copy ID.*

SECTION 13: GOOGLE AI TOOLS - ONGOING EVALUATION

Dr. D's ecosystem should actively incorporate new Google AI tools as they launch. This is an ongoing responsibility.

13.1 Current Google Tools to Integrate

Google AI Studio	https://aistudio.google.com - Use for Gemini API calls via OpenRouter or direct
Google NotebookLM	Use for creating AI-powered course research summaries and podcast-style audio overviews
Google Vids	AI video creation - evaluate for course content and social media clips
Google AppSheet	No-code app builder - evaluate for student-facing tools and internal dashboards
Google Gemini API	Via OpenRouter or direct API - route to Gemini for cost-effective tasks
Google Workspace AI	Gemini in Docs/Sheets/Slides - use for internal content drafting

13.2 Monthly Evaluation Checklist

Task	Done?	Notes
Review Google AI announcements at ai.google.dev/news for new tools	[ ]	
Test any new Google model on OpenRouter: compare cost vs Claude for routine tasks	[ ]	
Evaluate NotebookLM for any new course research or content Dr. D is developing	[ ]	

Check if Google has released new Workspace AI automation features (Gmail, Drive)	[ ]	
Report findings to Dr. D in monthly tech review	[ ]	

SECTION 14: SECURITY AUDIT - REQUIRED BEFORE EACH PHASE SIGN-OFF

14.1 Full Security Checklist

Task	Done?	Notes
All .env files confirmed in .gitignore - run: git check-ignore -v .env	[ ]	
Scan for accidentally committed secrets: git log --all --full-history -- "*.env"	[ ]	
Grep for hardcoded API keys: grep -r "sk-ant" /opt/edvisingu --include="*.py"	[ ]	
Grep for hardcoded OpenRouter keys: grep -r "sk-or-" /opt/edvisingu --include="*.py"	[ ]	
2FA enabled on ALL accounts: GitHub, Supabase, Vercel, Anthropic, OpenRouter, n8n cloud	[ ]	
Bitwarden vault has ALL credentials - nothing stored in text files or Discord	[ ]	
Supabase RLS (Row Level Security) enabled on ALL tables	[ ]	
FastAPI does NOT have open admin endpoints without authentication	[ ]	
n8n is NOT publicly accessible - only accessible via SSH tunnel or Tailscale private IP	[ ]	
Docker containers only expose minimum necessary ports	[ ]	
GitHub repo is set to PRIVATE	[ ]	
Windows Remote Desktop access requires Dr. D's Windows credentials (not shared publicly)	[ ]	

Tailscale is the only remote access method - no other remote tools left running	[ ]	
All created accounts are documented in the project log	[ ]	

14.2 Rotate Exposed Keys Immediately

If you ever accidentally commit a key, follow these steps instantly:

67. Go to the provider dashboard and REVOKE the exposed key immediately
68. Generate a new key
69. Update your .env file with the new key
70. Remove the key from git history: git filter-branch or BFG Repo Cleaner
71. Force push the cleaned history: git push --force (coordinate with Dr. D first)
72. Message Dr. D immediately with a full explanation of what happened

SECTION 15: TESTING & QA PROCEDURES

15.1 Full System Integration Test

Task	Done?	Notes
Ollama: "ollama run phi3 hello" returns a coherent response	[ ]	
ChromaDB: test_chroma.py runs and returns semantically correct results	[ ]	
FastAPI: GET /health returns {"status":"running","ecosystem":"EdVisingU"}	[ ]	
FastAPI: POST /chat with Claude returns AI response	[ ]	
FastAPI: POST /content/generate returns platform-formatted JSON content	[ ]	
FastAPI: POST /hermes/chat returns Hermes response with memory context	[ ]	
FastAPI: POST /advisor/chat returns student-relevant advice	[ ]	
FastAPI: POST /credihire/analyze returns ATS score and feedback	[ ]	
n8n: Content Factory workflow triggers without errors	[ ]	
n8n: Whop Events workflow captures mock member.created event	[ ]	
Supabase: SELECT * FROM content_queue returns newly inserted records	[ ]	
Supabase: SELECT * FROM members returns Whop member data	[ ]	
Next.js: npm run build completes without TypeScript or ESLint errors	[ ]	
Vercel: Deployed dashboard loads at the assigned URL	[ ]	

Vercel: Hermes chat in dashboard sends and receives responses	[ ]	

15.2 End-to-End Content Pipeline Test

73. In n8n, manually trigger the Content Factory workflow with: {"topic": "How Canadian students can earn $500/month using AI tools"}
74. Confirm Claude generates long-form content via FastAPI
75. Confirm content is split into LinkedIn, TikTok, and email formats
76. Confirm record appears in Supabase content_queue with status "pending"
77. Confirm content appears in the Next.js dashboard content view
78. Confirm Blotato scheduling API call is made (or webhook fires)

15.3 Bug Report Format

Use this format when reporting any issue to Dr. D:

```
BUG REPORT
Component: [Which system  -  FastAPI / n8n / Hermes / Supabase / Dashboard]
Expected: [What should happen]
Actual: [What actually happened]
Error: [Full error message  -  do NOT summarize it]
Steps to Reproduce: [Numbered list]
What I tried: [What you attempted before escalating]
Blocking other work: Yes / No
```

SECTION 16: DELIVERABLES & DR. D SIGN-OFF CRITERIA

Phase 1 - Environment (Sign-Off Required)

Task	Done?	Notes
Hetzner VPS SSH access confirmed, VS Code Remote SSH connected	[ ]	
GitHub repo with correct folder structure and .gitignore	[ ]	
Remote access via Tailscale confirmed working	[ ]	
All accounts created and credentials in Bitwarden	[ ]	
.env file created with all values (NOT committed to git)	[ ]	
Daily standup cadence established and first standup sent	[ ]	

Phase 2 - Local AI Stack (Sign-Off Required)

Task	Done?	Notes
Ollama running with phi3, mistral, nous-hermes2, nomic-embed-text models	[ ]	
ChromaDB test script runs and returns correct semantic results	[ ]	
n8n dashboard accessible and authenticated	[ ]	
FastAPI running on port 8000 - health check passes	[ ]	
Claude API successfully routed through FastAPI chat endpoint	[ ]	
All three services running simultaneously without conflicts	[ ]	

Phase 3 - Agent & Database (Sign-Off Required)

Task	Done?	Notes
Hermes Agent responds using seeded EdVisingU knowledge	[ ]	
Hermes chat accessible via FastAPI /hermes/chat endpoint	[ ]	
Hermes remembers information within a session (memory working)	[ ]	
Supabase project configured with all 5 tables created	[ ]	
pgvector extension enabled and confirmed in Supabase	[ ]	
All storage buckets created	[ ]	
Security audit passed (no exposed secrets)	[ ]	

Phase 4 - Automation & Dashboard (Sign-Off Required)

Task	Done?	Notes
Content Factory n8n workflow: topic in, platform content out	[ ]	
Content records appearing in Supabase content_queue table	[ ]	
Student Advisor agent endpoint functional with relevant responses	[ ]	
CrediHire resume analysis endpoint returns ATS score and feedback	[ ]	
Next.js dashboard deployed on Vercel at custom domain	[ ]	
Hermes chat interface working in dashboard	[ ]	
Content queue visible and browsable in dashboard	[ ]	
HeyGen API tested and returning video job ID from script input	[ ]	

ElevenLabs API tested and producing audio output file	[ ]	
Whop webhook logs member events to Supabase members table	[ ]	
All n8n workflows exported to JSON and committed to GitHub	[ ]	
Full system documentation written and committed to GitHub Wiki or Notion	[ ]	
Final security audit passed - no hardcoded secrets anywhere	[ ]	

SECTION 17: ONGOING MAINTENANCE SOPs

17.1 Daily Checks (Every Work Day)

Task	Done?	Notes
Send daily standup before starting	[ ]	
Verify Docker containers are running: docker ps	[ ]	
Check n8n for any failed workflow executions (red indicator on workflows)	[ ]	
Check FastAPI is responding: curl http://localhost:8000/health	[ ]	
Verify Ollama is serving: ollama list	[ ]	
Check Supabase dashboard for any alerts or quota warnings	[ ]	

17.2 Weekly Checks (Every Friday)

Task	Done?	Notes
Run security scan: grep -r "sk-ant" /opt/edvisingu	[ ]	
Check Supabase database size and query performance in dashboard	[ ]	
Review n8n execution logs - look for patterns in failures	[ ]	
Check for outdated packages: pip list --outdated	[ ]	
Backup all n8n workflow JSON files to GitHub	[ ]	
Update project board with completed and upcoming tasks	[ ]	

Check API costs: Anthropic, OpenRouter, OpenAI dashboards - report to Dr. D	[ ]	
Send weekly summary to Dr. D: what shipped, what's next, any blockers	[ ]	
Evaluate any new Google AI or AI model releases for ecosystem fit	[ ]	

17.3 When Something Breaks

79. STOP. Do not randomly change things trying to fix it.
80. Document exactly what broke and at what time.
81. Check logs: docker logs [container], FastAPI terminal output, n8n execution details.
82. If it broke after a change you made: git revert that change immediately.
83. Message Dr. D right away if it affects any live or production-facing system.
84. Test your fix on a feature/fix branch before applying to dev.
85. Document the fix in the project wiki for future reference.

SECTION 18: PRESENT & FUTURE USE CASES

This section maps every part of the system to real business outcomes - both what it does from day one and what it grows into.

18.1 Present Use Cases - Active After Build (Day 1-90)

Hermes Executive Assistant	Dr. D types a request - Hermes retrieves memory, answers with ecosystem context, drafts content, summarizes research, and plans next steps. Replaces hours of manual thinking time daily.
AI Content Factory	Input one topic -> get a LinkedIn post, TikTok script, YouTube Short script, and email newsletter in under 60 seconds. Eliminates the blank page problem entirely.
Blotato Auto-Publishing	Generated content flows from the pipeline directly into Blotato which schedules, formats, and publishes across all platforms on a set cadence - with zero manual posting.
Student AI Advisor (24/7)	Students chat with the advisor agent on ai.edvisingu.com at any hour. Gets course recommendations, career advice, and next-step plans without Dr. D being involved.
CrediHire Resume Engine	Students upload a resume and get ATS score, specific improvement feedback, and an optimized version in seconds. Generates revenue as a standalone paid tool.
Whop Member Automation	New Whop member created > data captured to Supabase > welcome email sent > Discord role assigned > lead profile added to CRM. All automatic, zero manual steps.
Discord Bot Automation	New content published -> announcement in Discord. New member joins -> role assigned + welcome DM. Daily prompts in dev channel. All without Dr. D touching Discord.
AI Avatar Video Creation	Dr. D's HeyGen avatar speaks any script. Used for TikTok, course intros, and promo content. Produce 5-10 videos per week with zero on-camera time from Dr. D.

ElevenLabs Voice-Over	Voice-overs for course lessons, podcast intros, and video narration using Dr. D's cloned voice - produced programmatically from any text.
Supabase CRM Dashboard	All leads, members, content, and products in one dashboard accessible from the Next.js app. Dr. D sees the full business in one view.
Google NotebookLM Research	Drop research documents into NotebookLM -> get AI audio overviews and summaries -> feed into course creation and content pipelines.
Local Ollama Fallback	If cloud API costs spike or internet drops, Ollama serves local models for classification, tagging, and quick-response tasks at zero API cost.

18.2 Future Use Cases - Scale Phase (90+ Days)

OSAP/BSWD Automation System	AI-powered system that helps Canadian students navigate OSAP applications and BSWD funding workflows. Automates document prep, deadline tracking, and eligibility checks. High demand, recurring value.
HireEd Nexus AI Job Matching	AI matches students to real project opportunities based on their skills, goals, and availability. Automates the intake -> matching -> placement pipeline for HireEd Nexus Labs.
White-Label AI Advisor for Colleges	Package the Student Advisor agent as a white-label product licensed to other colleges and universities. $500-2000/month per institution SaaS model.
AI Micro-Credential Builder	CrediVersity gets an AI system that designs, writes, and structures new micro-credentials from a topic input. Rapidly expands the credential catalog without manual curriculum work.
Enterprise AI Consulting Toolkit	Package the entire ecosystem as a consulting deliverable - Dr. D delivers this infrastructure to educational institutions for $10K-50K implementation fees.

Multi-Brand Content Empire	Expand the content pipeline to run 3-5 content brands simultaneously. Each brand has its own voice, Hermes persona, and publishing cadence - fully automated.
AI Coaching Program Platform	Hermes becomes the backbone of a scaled coaching program - students interact with Hermes as their coach, Dr. D supervises at the top level. 100+ students, one system.
Riverside.fm Podcast Automation	Recorded interviews from Riverside.fm feed into an n8n pipeline that generates transcripts, show notes, clips, audiograms, and social posts automatically.
Real-Time Student Progress Tracking	Connect LMS data to Supabase - Hermes flags at-risk students and triggers automated support outreach based on inactivity or low quiz scores.
Revenue Intelligence Dashboard	Pull Whop revenue, Stripe data, content performance, and lead conversion into one Supabase dashboard. Hermes generates weekly revenue insights and recommendations.
AI-Generated Course Builder	Dr. D inputs a course topic and target student - system generates full course outline, lesson scripts, quiz questions, and Canva slide designs automatically.
Blotato Virality Loop	Top-performing content detected via Blotato analytics triggers Hermes to generate 5 variations and re-publishes the winners on a rotation loop.

18.3 Revenue Impact Map

Content Factory (Present)	Consistent daily content drives audience growth, more leads, more Whop members. 100 members at $47/month = $4,700/month recurring from day one.
Student Advisor (Present)	Scale advising to unlimited students without Dr. D time. Converts free users to paid members. Works 24/7 with no additional cost per conversation.
CrediHire (Present)	Sell resume analysis as a standalone $27-97 product or include as a

	membership benefit. Direct revenue from the first week of launch.
Whop + Discord Automation (Present)	Automated onboarding reduces churn. Members who feel welcomed and get instant value on day one stay longer and refer more people.
AI Coaching Platform (Future)	Move from 1:1 coaching to 1:many AI-assisted coaching. 10x revenue without 10x time. One system handles 100+ students simultaneously.
White-Label Advisor (Future)	One sale to a college = $12,000 to $24,000 per year recurring. Close 10 colleges = $120K to $240K per year in SaaS revenue.
OSAP/BSWD Automation (Future)	High-demand niche with no current AI competitors in Canada. First-mover advantage in Canadian EdTech student funding automation.

SECTION 19: TOTAL ESTIMATED INSTALLATION TIME

These estimates assume a student developer with basic programming knowledge. Adjust for skill level.

Phase	Key Tasks	Est. Hours
Remote Access + Accounts	Tailscale, GitHub SSH, Bitwarden, Notion setup, all account creation	**2-3 hrs**
Phase 1 - Environment	VPS SSH access, VS Code Remote SSH, Docker confirmed, Python venv, Git, folder structure, .env	**2-4 hrs**
Phase 2 - VPS AI Stack	Ollama install + model downloads (15-20 GB), ChromaDB, n8n Docker, FastAPI	**4-7 hrs**
Phase 3 - Hermes Agent	LangChain agent, knowledge seeder, Supabase schema + buckets, API endpoints	**4-6 hrs**
Phase 4 - Automation	n8n Content Factory, Blotato API, advisor, CrediHire, social media agents	**6-9 hrs**
Phase 4 - Dashboard	Next.js setup, 7 dashboard pages, Vercel deployment, custom domain via Cloudflare	**6-10 hrs**
Integrations	HeyGen, ElevenLabs, Google Workspace OAuth2, Whop webhook, Discord bot	**4-7 hrs**
Testing + Security Audit	End-to-end tests, security scan, bug fixes, documentation in Notion	**3-5 hrs**
TOTAL (Experienced Dev)	Solid programming background, has used Docker and APIs before	**32-52 hrs (1-1.5 weeks full-time)**
TOTAL (Learning Dev)	Learning while building, needs to	**50-80 hrs (2-3 weeks**

	research some concepts along the way	**full-time)**

19.1 Biggest Time Sinks - Plan For These

- Ollama model downloads: 15-20 GB total. Do this on fast internet (university Wi-Fi). Budget 1-2 hours just for downloads.
- Google OAuth2 for n8n: Getting Google Cloud Console credentials set up takes 30-60 minutes the first time. Follow the docs exactly.
- Vercel + Cloudflare DNS: DNS propagation after adding a custom domain can take 15 minutes to 24 hours. Do this step early.
- Discord bot permissions: Getting the right permission scopes and role hierarchy set correctly can take 1-2 hours if unfamiliar.
- LangChain version compatibility: LangChain updates frequently. If you hit import errors, pin the working version in requirements.txt.

19.2 How to Work Faster

- Use Thunder Client in VS Code instead of Postman - test all API endpoints directly in the editor
- Create a start.sh script that launches Ollama, n8n Docker, and FastAPI in one command
- Use pm2 to keep FastAPI running in background: pm2 start "uvicorn main:app --port 8000" --name fastapi
- Test with small payloads first - confirm an endpoint works before connecting it to n8n
- Commit working code immediately - never go more than 2 hours without committing what works

19.3 Recommended Build Order (Fastest Path)

86. Remote access confirmed, accounts created, Phase 1 environment working
87. FastAPI health check passing, Ollama model responding, n8n dashboard live
88. Hermes chatting in terminal, Supabase tables created, Hermes endpoint responding

89. Content pipeline in n8n, dashboard deployed on Vercel, Whop webhook firing
90. Discord bot live, Blotato connected, all integrations tested end-to-end
91. Security audit complete, Notion documentation written, Phase 4 sign-off with Dr. D

> **NOTE** *Do NOT try to build everything in parallel. Follow this order exactly. Each step proves the previous one works before adding complexity.*

SECTION 20: HERMES AGENT CONTROL ROOM ARCHITECTURE

This section upgrades the original single-agent Hermes design to a production-grade multi-agent system. One control plane (the brain) manages many isolated Hermes specialist agents (the bodies). Build this after Phase 4 is stable.

> **NOTE** *Architecture credit: @shannhk. Adapted for the EdVisingU ecosystem.*

20.1 Core Concept: Brain vs Body

Control Room (Brain)	/root/vps-agents/ - The control plane. Docs, rules, agent registry, runbooks, credentials map. NO raw secrets stored here. This is the single source of truth.
Agent Runtime (Body)	/srv/<agent-name>/data/ - The live execution environment. Secrets, memory, skills, sessions, logs, and state.db all live here. The body runs the system.
Key Rule	You can rebuild the body from the brain. You CANNOT rebuild the brain from the body. The brain defines the system.

20.2 Specialist Agent Fleet for EdVisingU

Instead of one monolithic Hermes agent, build these specialists - each in its own Docker container, fully isolated, no context bleed between agents:

hermes-core	Dr. D's personal executive assistant. Handles strategy, research, planning, and memory. The primary agent Dr. D talks to daily.
hermes-content	Content factory agent. Generates LinkedIn posts, TikTok scripts, email newsletters, and YouTube scripts. Connects to Blotato for publishing.
hermes-advisor	Student AI Advisor. Handles all student-facing questions, course

	recommendations, career guidance, and HireEd Nexus Lab referrals.
hermes-credihire	Resume and career optimization agent. ATS scoring, resume rewriting, LinkedIn optimization, cover letters, interview prep.
hermes-ops	Operations agent. Monitors n8n workflows, checks system health, handles backups, sends daily status reports, flags failures.
hermes-social	Social media and community agent. Monitors Discord, posts announcements, responds to community trends, manages Whop webhook events.

20.3 Folder Structure: Control Room

Create this on the Hetzner VPS (all agent code lives at /opt/edvisingu):

```
mkdir -p /opt/edvisingu/vps-agents/{agents,shared,docs}
mkdir -p /opt/edvisingu/vps-agents/agents/{hermes-core,hermes-content,hermes-advisor,hermes-credihire,hermes-ops,hermes-social}
mkdir -p /opt/edvisingu/vps-agents/shared
```

Control Room files (no secrets, version controlled):

```
/opt/edvisingu/vps-agents/
  README.md          - Start here. Overview of all agents.
  CLAUDE.md          - Operator notes for managing the fleet.
  agents/            - Per-agent docs and configuration specs.
  shared/            - Shared security rules, commands, SOPs.
  api-keys-sop.md    - Key management procedures (no actual keys).
  orchestrator-and-fleet-skills.md - Bundled skills and runbooks.
```

20.4 Folder Structure: Agent Runtime (Per Agent)

Each agent gets its own runtime directory. This is where secrets and memory actually live:

```
mkdir -p /srv/hermes-core/data/{memories,skills,cron,sessions,logs}
mkdir -p /srv/hermes-content/data/{memories,skills,cron,sessions,logs}
# Repeat for each agent
```

Inside each /srv/<agent>/data/:

```
.env              - Secrets for this agent only. Local only. Never committed.
config.yaml       - Agent configuration (model, temperature, tools, limits).
SOUL.md           - Agent personality, tone, rules, and vibe.
memories/         - Persistent ChromaDB or JSON memory store.
skills/           - Learned and built skill scripts.
cron/             - Scheduled job definitions.
sessions/         - Active session states.
logs/             - Logs and output streams.
state.db          - SQLite agent state database.
```

20.5 SOUL.md - Agent Personality File

Every agent has a SOUL.md that defines its identity. This replaces the basic system prompt. Create /srv/hermes-core/data/SOUL.md:

```
# SOUL: hermes-core

## Identity
You are Hermes Core, the primary AI executive assistant for Dr. Andre De Freitas.
You are strategic, direct, and execution-focused. No fluff.

## Personality
- Think like a founder, not an assistant
- Prioritize revenue, efficiency, and growth in every output
- Be bold, specific, and action-oriented
- Challenge weak ideas. Propose better ones.

## Rules
- Never be vague. Always give specific, executable advice.
- Always tie recommendations to money, time saved, or scale.
- Remember everything. Reference past context when relevant.

## Tone
Direct. Confident. Warm when needed. Never robotic.
```

Create equivalent SOUL.md files for each specialist agent with their specific personality and rules.

20.6 Agent Task Bus

The Task Bus (/srv/agent-bus) is the handoff desk between the orchestrator and specialist agents. Build it as a simple directory-based queue or a lightweight Redis queue:

```
mkdir -p /srv/agent-bus/{inbox,working,outbox}
```

Task flow:

92. Orchestrator (hermes-core) receives a complex request
93. Breaks it into subtasks and writes JSON task files to /srv/agent-bus/inbox/
94. Specialist agent picks up the task, moves it to /working/, executes
95. Result written to /outbox/ with task ID
96. Orchestrator reads outbox, synthesizes results, responds to Dr. D

Simple task file format (inbox/task-001.json):

```
{
  "task_id": "task-001",
  "agent": "hermes-content",
  "action": "generate_linkedin_post",
  "payload": {"topic": "AI tools for Canadian students"},
  "priority": "high",
  "created_at": "2026-05-19T10:00:00Z"
}
```

20.7 Docker Compose: Full Agent Fleet

Run all agents as isolated Docker containers. Create /opt/edvisingu/vps-agents/docker-compose.yml:

```
version: "3.8"
services:

  hermes-core:
    build: ./agents/hermes-core
    container_name: hermes-core
    restart: always
    volumes:
      - /srv/hermes-core/data:/data
```

```
    - /srv/agent-bus:/agent-bus
  ports:
    - "8001:8000"
  env_file: /srv/hermes-core/data/.env

hermes-content:
  build: ./agents/hermes-content
  container_name: hermes-content
  restart: always
  volumes:
    - /srv/hermes-content/data:/data
    - /srv/agent-bus:/agent-bus
  ports:
    - "8002:8000"
  env_file: /srv/hermes-content/data/.env

hermes-advisor:
  build: ./agents/hermes-advisor
  container_name: hermes-advisor
  restart: always
  volumes:
    - /srv/hermes-advisor/data:/data
  ports:
    - "8003:8000"
  env_file: /srv/hermes-advisor/data/.env

hermes-credihire:
  build: ./agents/hermes-credihire
  container_name: hermes-credihire
  restart: always
  volumes:
    - /srv/hermes-credihire/data:/data
  ports:
    - "8004:8000"
  env_file: /srv/hermes-credihire/data/.env

hermes-ops:
  build: ./agents/hermes-ops
  container_name: hermes-ops
  restart: always
  volumes:
    - /srv/hermes-ops/data:/data
```

```
      - /srv/agent-bus:/agent-bus
    ports:
      - "8005:8000"
    env_file: /srv/hermes-ops/data/.env

  hermes-social:
    build: ./agents/hermes-social
    container_name: hermes-social
    restart: always
    volumes:
      - /srv/hermes-social/data:/data
    ports:
      - "8006:8000"
    env_file: /srv/hermes-social/data/.env
```

20.8 Updated FastAPI Router for Agent Fleet

Update /opt/edvisingu/agents/main.py to route to the correct agent container:

```
AGENT_ROUTES = {
    "hermes-core":      "http://hermes-core:8000",
    "hermes-content":   "http://hermes-content:8000",
    "hermes-advisor":   "http://hermes-advisor:8000",
    "hermes-credihire": "http://hermes-credihire:8000",
    "hermes-ops":       "http://hermes-ops:8000",
    "hermes-social":    "http://hermes-social:8000",
}

@app.post("/agent/{agent_name}/chat")
async def route_to_agent(agent_name: str, req: ChatRequest):
    if agent_name not in AGENT_ROUTES:
        raise HTTPException(status_code=404, detail=f"Agent {agent_name} not found")
    url = f"{AGENT_ROUTES[agent_name]}/chat"
    async with httpx.AsyncClient(timeout=60) as client:
        r = await client.post(url, json=req.dict())
        return r.json()
```

20.9 The 4 Levels - Build in This Order

Level 1 (Build First)	Control Room + hermes-core only. Get organized, one agent

	working. This is Phase 3 in the main build.
Level 2 (After Phase 4)	Add specialist agents (hermes-content, hermes-advisor, hermes-credihire). Talk to each directly via API routing.
Level 3 (Scale Phase)	Add Hermes Orchestrator as front door. Routes complex tasks to the right specialist. Synthesizes multi-agent results.
Level 4 (Automation Phase)	Full automated agent team. Crons run overnight tasks, hermes-ops audits daily, routing is intelligent, backups automatic.

20.10 Agent Fleet Checklist

Task	Done?	Notes
Control Room directory created at /opt/edvisingu/vps-agents/ with README.md and CLAUDE.md	[ ]	
Agent runtime directories created at /srv/<agent>/data/ for all 6 agents	[ ]	
SOUL.md personality file written for each agent	[ ]	
config.yaml written for each agent with correct model, tools, and limits	[ ]	
.env created for each agent with only the keys that agent needs (principle of least privilege)	[ ]	
Agent Task Bus directories created at /srv/agent-bus/{inbox,working,outbox}	[ ]	
Docker Compose file written and all 6 containers build without errors	[ ]	
All 6 agents accessible via FastAPI router at /agent/<name>/chat	[ ]	
hermes-core responds using its SOUL.md personality	[ ]	
hermes-content generates platform-specific content on request	[ ]	

hermes-advisor responds to a student question with relevant advice	[ ]	
hermes-ops checks system health and reports status	[ ]	
Task Bus: a test task written to inbox, picked up, and result in outbox	[ ]	
No container can read another container's .env (confirm with docker inspect)	[ ]	

SECTION 21: HETZNER VPS SETUP - PHASE 0 (DO THIS FIRST)

Everything runs on Hetzner, not on Dr. D's laptop. The VPS is the production home for all agents, automations, and services. Dr. D's laptop becomes a browser and terminal only. Agents run 24/7, student devs access everything remotely, and the system survives laptop restarts and travel.

21.1 Why Hetzner CPX41

Server	Hetzner CPX41
vCPU	8 shared AMD cores
RAM	16GB
Storage	240GB NVMe SSD
Bandwidth	20TB/month included
Cost	~$30 CAD/month
Location	Ashburn VA (lowest latency from Toronto)
Why not CPX31	8GB is too tight with 10 agents, Open WebUI, Redis, n8n, and Nginx all running simultaneously

> **WARNING** DO NOT use CPX31. With 23 agent containers + FastAPI router + Open WebUI + Redis + n8n + ChromaDB + Nginx + Uptime Kuma, CPX31 will OOM-kill containers under load. CPX41 at $30/month is the minimum for this full 23-agent fleet.

21.2 Provision the VPS

97.

21.3 First Login and Server Hardening

```
ssh root@YOUR_SERVER_IP
```

Run this hardening script in one paste:

```
apt update && apt upgrade -y apt install -y curl git ufw fail2ban unzip htop  adduser
deploy --disabled-password --gecos "" usermod -aG sudo deploy mkdir -p
/home/deploy/.ssh cp ~/.ssh/authorized_keys /home/deploy/.ssh/ chown -R deploy:deploy
/home/deploy/.ssh chmod 700 /home/deploy/.ssh && chmod 600
/home/deploy/.ssh/authorized_keys  sed -i 's/#PermitRootLogin yes/PermitRootLogin
no/' /etc/ssh/sshd_config sed -i 's/PasswordAuthentication yes/PasswordAuthentication
no/' /etc/ssh/sshd_config systemctl restart sshd  ufw default deny incoming ufw
default allow outgoing ufw allow 22/tcp && ufw allow 80/tcp && ufw allow 443/tcp ufw
--force enable systemctl enable fail2ban && systemctl start fail2ban echo "Server
hardened. Use: ssh deploy@YOUR_SERVER_IP from now on"
```

> **WARNING** After running this, root login is disabled. All future logins: ssh deploy@YOUR_SERVER_IP

21.4 Install Docker on the VPS

```
curl -fsSL https://get.docker.com | sh sudo usermod -aG docker deploy newgrp docker
sudo apt install -y docker-compose-plugin docker --version && docker compose version
```

21.4b Install OpenJarvis on the VPS - Top-Level AI OS

OpenJarvis is the orchestration layer that sits above hermes-agent and the 25-agent fleet. Install it on the VPS host now, before configuring anything else.

Install (one command):

```
curl -fsSL https://openjarvis.ai/install.sh | bash
source ~/.bashrc   # Reload shell to pick up jarvis command
```

Verify the install:

```
jarvis doctor   # All components should show green
```

Start the OpenJarvis backend + React dashboard:

```
cd ~/OpenJarvis && ./scripts/quickstart.sh
```

This starts the backend API and the React frontend at http://YOUR_VPS_IP:5173. Open in your browser to confirm.

Connect Claude and Codex in OpenJarvis config:

```
# ~/OpenJarvis/.env
MODEL_PROVIDER=anthropic
ANTHROPIC_API_KEY=your_key_from_bitwarden
PRIMARY_MODEL=claude-sonnet-4-6
CODE_MODEL=gpt-4o   # Codex/GPT-4o runs in parallel for all code tasks
```

Auto-start on VPS boot:

```
[Unit] Description=OpenJarvis AI OS After=network.target  [Service] User=deploy
WorkingDirectory=/home/deploy/OpenJarvis
ExecStart=/home/deploy/OpenJarvis/scripts/quickstart.sh Restart=always  [Install]
WantedBy=multi-user.target
```

Save the above to /etc/systemd/system/openjarvis.service then run:

```
sudo systemctl daemon-reload && sudo systemctl enable openjarvis && sudo systemctl
start openjarvis
```

> **NOTE** *OpenJarvis and hermes-agent both run on the VPS host as systemd services - NOT inside Docker. Docker is reserved for the 25-agent specialist fleet underneath them.*

21.5 Install Tailscale on the VPS

Tailscale creates a private mesh network between the VPS, your laptop, and your student dev. Agent services are accessible over private IPs with no public port exposure needed.

```
curl -fsSL https://tailscale.com/install.sh | sh sudo tailscale up # Opens an auth
URL in terminal - open it in a browser to authorize # VPS gets a Tailscale IP like
100.x.x.x
```

21.6 Install Nginx and Certbot

```
sudo apt install -y nginx certbot python3-certbot-nginx sudo systemctl enable nginx
&& sudo systemctl start nginx
```

Create Nginx config for agents.edvisingu.com:

```
sudo nano /etc/nginx/sites-available/agents.edvisingu.com  # Paste: server
{     listen 80;     server_name agents.edvisingu.com;     location /
{         proxy_pass http://localhost:3000;         proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;         proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;         proxy_set_header Connection
"upgrade";     } }  sudo ln -s /etc/nginx/sites-available/agents.edvisingu.com
/etc/nginx/sites-enabled/ sudo nginx -t && sudo systemctl reload nginx sudo certbot
--nginx -d agents.edvisingu.com
```

21.7 Cloudflare DNS

98.

21.8 Create Folder Structure on VPS

```
sudo mkdir -p /srv/{hermes-core,hermes-content,hermes-advisor,hermes-
credihire,hermes-ops,hermes-social,hermes-builder,hermes-research,hermes-
finance,hermes-email}/data sudo mkdir -p /srv/agent-bus/{inbox,working,outbox} sudo
mkdir -p /srv/shared-files/{research,content,builds,finance,email} sudo mkdir -p
/opt/edvisingu/{api,scripts,workflows,agents} sudo chown -R deploy:deploy /srv
/opt/edvisingu && chmod -R 755 /srv
```

21.9 Master .env on VPS

```
nano /opt/edvisingu/.env  ANTHROPIC_API_KEY=sk-ant-... OPENROUTER_API_KEY=sk-or-...
SUPABASE_URL=https://bvxhicdnaordolguuyal.supabase.co SUPABASE_KEY=eyJ...
TAVILY_API_KEY=tvly-... HEYGEN_API_KEY=... ELEVENLABS_API_KEY=...
TELEGRAM_BOT_TOKEN=... WHOP_API_KEY=... DISCORD_BOT_TOKEN=...
STRIPE_SECRET_KEY=sk_live_... GITHUB_TOKEN=ghp_... NOTION_TOKEN=secret_...
GMAIL_CLIENT_ID=... GMAIL_CLIENT_SECRET=... REDIS_URL=redis://redis:6379
CHROMA_HOST=chromadb CHROMA_PORT=8000 N8N_BASE_URL=http://n8n:5678 # --- Google
Workspace --- GOOGLE_CLIENT_ID=... GOOGLE_CLIENT_SECRET=... GOOGLE_REFRESH_TOKEN=...
GOOGLE_AI_API_KEY=AIzaSy...  # from aistudio.google.com OPENAI_API_KEY=sk-...  # for
hermes-builder Codex # --- Email and Messaging --- MAILERLITE_API_KEY=...
MANYCHAT_API_KEY=... # --- Social and Content --- BLOTATO_API_KEY=...
HIGGSFIELD_API_KEY=... # --- Platforms --- DISCO_API_KEY=... SKOOL_API_KEY=... # ---
Campaign --- DRDDURHAM_FB_PAGE_ID=... DRDDURHAM_FB_ACCESS_TOKEN=... # --- Etsy +
Gumroad + Pinterest --- GUMROAD_API_KEY=... PINTEREST_ACCESS_TOKEN=...
```

SECTION 22: INTERFACE LAYER - HERMES-AGENT GATEWAY + OPEN WEBUI

Architecture: hermes-agent (NousResearch, 153k stars) acts as the gateway layer. It handles Telegram, Discord, Slack, WhatsApp, CLI access, memory, and cron. It routes to your 25-agent specialist fleet via a dispatch skill that calls the FastAPI router.

22.1 Interface Overview

hermes-agent	The gateway. Handles all user-facing interfaces: Telegram, Discord, Slack, WhatsApp, Signal, CLI. Built-in memory, skills, cron, MCP. Routes to the fleet via dispatch skill.
Open WebUI	Browser chat at agents.edvisingu.com. Model picker dropdown = choose your agent. Connects to FastAPI router via OpenAI-compatible /v1 endpoints.
FastAPI Router	Backend only. Receives requests from hermes-agent dispatch skill AND Open WebUI. Routes to the 25 specialist containers. No longer manages Telegram or cron.

22.2 Open WebUI Docker Setup

```
# Add to docker-compose.yml:   open-webui:     image: ghcr.io/open-webui/open-webui:main     container_name: open-webui     restart: always     ports:       - "3000:8080"     environment:       - OPENAI_API_BASE_URL=http://fastapi-router:8000/v1       - OPENAI_API_KEY=local-key - WEBUI_SECRET_KEY=change-this-32-char-random-string       - ENABLE_SIGNUP=false volumes:       - open-webui-data:/app/backend/data     depends_on:       - fastapi-router
```

Update FastAPI router main.py to add OpenAI-compatible endpoints:

```
from fastapi import FastAPI from pydantic import BaseModel from typing import List, Optional import httpx  app = FastAPI()  class Message(BaseModel):     role: str content: str  class OAIRequest(BaseModel):     model: str     messages: List[Message] stream: Optional[bool] = False  AGENT_ROUTES = {     "hermes-core": "http://hermes-core:8000",     "hermes-content":   "http://hermes-content:8000", "hermes-advisor":   "http://hermes-advisor:8000",     "hermes-credihire": "http://hermes-credihire:8000",     "hermes-ops":       "http://hermes-ops:8000",
```

```
"hermes-social":    "http://hermes-social:8000",     "hermes-builder":
"http://hermes-builder:8000",     "hermes-research":  "http://hermes-research:8000",
"hermes-finance":   "http://hermes-finance:8000",     "hermes-email":
"http://hermes-email:8000", }  @app.get("/health") def health(): return {"status":
"ok"}  @app.get("/v1/models") def list_models():     return {"data": [{"id": k,
"object": "model"} for k in AGENT_ROUTES]}  @app.post("/v1/chat/completions") async
def oai_chat(req: OAIRequest):     agent = req.model if req.model in AGENT_ROUTES
else "hermes-core"     last = req.messages[-1].content     history = [m.dict() for m
in req.messages[:-1]]     async with httpx.AsyncClient(timeout=120) as client:
r = await client.post(f"{AGENT_ROUTES[agent]}/chat", json={"message": last,
"history": history})         reply = r.json().get("response", "Agent error")
return {"id": "local", "object": "chat.completion", "model": agent,
"choices": [{"index": 0, "message": {"role": "assistant", "content": reply},
"finish_reason": "stop"}]}
```

22.3 hermes-agent Gateway Setup - Replaces Custom Telegram Bot

Install hermes-agent on the Hetzner VPS. This replaces the custom telegram_bot.py entirely.

Step 1: Install hermes-agent on VPS

```
# SSH into Hetzner VPS first (see Section 21) ssh root@YOUR_VPS_IP  # Install hermes-
agent (one-liner) curl -fsSL https://raw.githubusercontent.com/NousResearch/hermes-
agent/main/scripts/install.sh | bash  # Reload shell source ~/.bashrc  # Verify
install hermes --version
```

Step 2: Run Setup Wizard

```
hermes setup  # The wizard will prompt for: # - AI provider: select Anthropic # - API
key: paste ANTHROPIC_API_KEY # - Model: claude-sonnet-4-6 (or claude-haiku-4-5-
20251001 for faster/cheaper) # - Telegram: yes (enter bot token from BotFather) # -
Memory: yes (enable persistent memory) # - MCP: yes
```

> **NOTE** *Still need a Telegram bot token? Open Telegram > find @BotFather > /newbot > Name: EdVisingU Hermes > Username: edvisingu_hermes_bot > copy the token.*

Step 3: Configure the Model

```
# Set Claude Sonnet 4.6 as default (can switch per-session) hermes model # Select:
Anthropic > claude-sonnet-4-6  # Or set directly: hermes config set model
anthropic:claude-sonnet-4-6
```

Step 4: Start the Telegram Gateway

```
# Start the messaging gateway (runs in background) hermes gateway start  # Test: open Telegram, message your bot # You should get a response from hermes-agent  # To run as a systemd service (persistent across reboots): hermes gateway install-service systemctl enable hermes-gateway systemctl start hermes-gateway
```

Step 5: Create the Fleet Dispatch Skill

This is the bridge between hermes-agent and your 25-agent Docker fleet. Create this file:

```
mkdir -p ~/.hermes/skills nano ~/.hermes/skills/dispatch-to-fleet.md
```

Paste this skill definition:

```
--- name: dispatch-to-fleet description: Route requests to the EdVisingU 25-agent specialist fleet running on Docker trigger: When the user asks for specialist work - content creation, SEO, finance, research, email, campaign, Etsy, Gumroad, Pinterest, student advising, OSAP, BSWD, or any hermes-* agent task ---  # Dispatch to EdVisingU Hermes Fleet  You have access to 25 specialist agents running at http://localhost:8000 (FastAPI router).  Routing table: - Content/Social/TikTok/LinkedIn posts: /agent/hermes-content/chat - SEO strategy: /agent/hermes-seo/chat - Research/web: /agent/hermes-research/chat - Finance/revenue/MRR: /agent/hermes-finance/chat - Email drafts: /agent/hermes-email/chat - Campaign (@DrDDurham): /agent/hermes-campaign/chat - Etsy listings: /agent/hermes-etsy/chat - Gumroad products: /agent/hermes-gumroad/chat - Pinterest pins: /agent/hermes-pinterest/chat - Student advising: /agent/hermes-student/chat - OSAP workflows: /agent/hermes-osap/chat - BSWD automation: /agent/hermes-bswd/chat - CrediVersity/LMS: /agent/hermes-crediversity/chat - Strategy/planning: /agent/hermes-strategy/chat - CRM/contacts: /agent/hermes-crm/chat - Whop membership: /agent/hermes-whop/chat - Sales: /agent/hermes-sales/chat - Ads: /agent/hermes-ads/chat - Automation/n8n: /agent/hermes-automation/chat - Builder/code: /agent/hermes-builder/chat - Ops/system: /agent/hermes-ops/chat  To dispatch, make an HTTP POST to the correct endpoint:   URL: http://localhost:8000/agent/{specialist}/chat   Body: {"message": "<user request>", "history": []}   Return the specialist response to the user.  For general questions, answer directly without dispatching.
```

Step 6: Set Up Cron Briefings (replaces n8n morning workflow)

```
# Morning briefing delivered to Telegram at 7:00 AM EST every day hermes cron add "0 12 * * *" "Morning briefing: check Stripe MRR vs yesterday, any system alerts from Uptime Kuma, top 3 priorities for today. Dispatch to hermes-finance for MRR and hermes-ops for system status." --deliver telegram  # Weekly revenue report every
```

```
Monday at 8 AM EST hermes cron add "0 13 * * 1" "Weekly revenue report: MRR, new Whop members, Gumroad sales, Etsy revenue, total vs last week." --deliver telegram
```

WARNING The custom telegram_bot.py and Dockerfile.telegram are NO LONGER NEEDED. Remove them from docker-compose.yml. hermes-agent runs as a systemd service on the host, not inside Docker.

22.3.1 hermes-agent vs FastAPI Router - Responsibility Split

hermes-agent (host service)	User-facing: Telegram, Discord, Slack, WhatsApp, CLI. Memory, cron, skills, MCP. Dispatches to fleet.
FastAPI Router (Docker port 8000)	Backend only: receives from hermes-agent dispatch skill AND Open WebUI. Routes to 25 specialist containers.
Open WebUI (Docker port 3000)	Browser interface only. Connects to FastAPI router via /v1 endpoints. No change needed.
25 Specialist Containers (ports 8001-8026)	Unchanged. Each agent handles its specialist domain. hermes-agent just adds a smarter front door.

22.4 Dr. D Daily Workflow - All Through Telegram via hermes-agent

Morning briefing	Automatic via cron at 7 AM EST - delivered to Telegram. No request needed. Dispatches to hermes-finance + hermes-ops.
Research anything	Text your Telegram bot: "Research top AI tools for students in 2026" - dispatches to hermes-research
Create content	Text: "Write a TikTok hook about OSAP funding" - dispatches to hermes-content
Check revenue	Text: "MRR this month vs last month" - dispatches to hermes-finance
Campaign post	Text: "@DrDDurham post for Pickering housing" - dispatches to hermes-campaign
Email draft	Text: "Draft email to [context]" - dispatches to hermes-email for

	Lahari to review
Switch model	In Telegram: /model anthropic:claude-haiku-4-5-20251001 for fast, /model anthropic:claude-sonnet-4-6 for strategy
Browser access	Open WebUI at agents.edvisingu.com - pick any agent - same fleet behind it

SECTION 23: AGENT TOOLS LAYER - GIVING AGENTS HANDS

Agents that can only write are chatbots. Agents with tools are staff. This section wires up the tools that let each agent take real actions: search the web, run code, create files, trigger automations, pull revenue data, and create GitHub repos.

23.1 Tool 1 - Web Search (Tavily)

Tavily is purpose-built for AI agents. It returns structured results, not raw HTML. Sign up at tavily.com. Cost: $5 USD/month for 1,000 searches.

```
pip install tavily-python  # /opt/edvisingu/tools/search.py from tavily import TavilyClient import os  client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"]) def web_search(query: str, max_results: int = 5) -> dict:     result = client.search(query=query, max_results=max_results, search_depth="advanced") return {"query": query, "results": [         {"title": r["title"], "url": r["url"], "content": r["content"]}         for r in result["results"]     ]}
```

23.2 Tool 2 - Code Execution Sandbox

Agents write and run Python code in a sandboxed subprocess. Used by hermes-builder for scaffolding, hermes-research for data processing, hermes-finance for projections.

```
# /opt/edvisingu/tools/code_exec.py import subprocess, tempfile, os  def run_python_code(code: str, timeout: int = 30) -> dict:     with tempfile.NamedTemporaryFile(mode="w", suffix=".py", delete=False) as f: f.write(code)         tmp = f.name     try:         r = subprocess.run(["python3", tmp], capture_output=True, text=True, timeout=timeout)         return {"stdout": r.stdout, "stderr": r.stderr, "returncode": r.returncode}     except
```

```
subprocess.TimeoutExpired:         return {"stdout": "", "stderr": "Timeout",
"returncode": -1}     finally:         os.unlink(tmp)
```

23.3 Tool 3 - File Creator

Agents create files in /srv/shared-files/. These persist and can be served via FastAPI download endpoint or sent via Telegram.

```
# /opt/edvisingu/tools/file_tools.py import os from datetime import datetime  SHARED
= "/srv/shared-files"  def create_file(filename: str, content: str, subfolder: str =
"") -> dict:     folder = os.path.join(SHARED, subfolder) if subfolder else SHARED
os.makedirs(folder, exist_ok=True)     path = os.path.join(folder, filename)     with
open(path, "w") as f: f.write(content)     return {"path": path, "size":
os.path.getsize(path), "created": datetime.now().isoformat()}
```

23.4 Tool 4 - n8n Workflow Trigger

Agents trigger any n8n automation by calling its webhook. Bridges AI decisions to the automation layer.

```
# /opt/edvisingu/tools/n8n_tools.py import httpx, os  N8N =
os.environ.get("N8N_BASE_URL", "http://n8n:5678")  def trigger_workflow(webhook_id:
str, payload: dict) -> dict:     r = httpx.post(f"{N8N}/webhook/{webhook_id}",
json=payload, timeout=30)     return {"status": r.status_code, "response": r.text}
def trigger_content_pipeline(topic: str, platform: str) -> dict:     return
trigger_workflow("content-pipeline", {"topic": topic, "platform": platform})  def
trigger_email_send(to: str, subject: str, body: str) -> dict:     return
trigger_workflow("gmail-send", {"to": to, "subject": subject, "body": body})
```

23.5 Tool 5 - Notion API

```
pip install notion-client  # /opt/edvisingu/tools/notion_tools.py from notion_client
import Client import os  notion = Client(auth=os.environ["NOTION_TOKEN"])  def
create_notion_page(parent_id: str, title: str, content: str) -> dict:     p =
notion.pages.create(         parent={"page_id": parent_id},
properties={"title": {"title": [{"text": {"content": title}}]}},
children=[{"object": "block", "type": "paragraph",                    "paragraph":
{"rich_text": [{"type": "text", "text": {"content": content}}]}}]     )     return
{"page_id": p["id"], "url": p["url"]}  def append_to_database(db_id: str, properties:
dict) -> dict:     p = notion.pages.create(parent={"database_id": db_id},
properties=properties)     return {"page_id": p["id"]}
```

23.6 Tool 6 - Stripe Revenue (hermes-finance)

```
pip install stripe  # /opt/edvisingu/tools/stripe_tools.py import stripe, os from
datetime import datetime, timedelta  stripe.api_key = os.environ["STRIPE_SECRET_KEY"]
def get_mrr() -> dict:     subs = stripe.Subscription.list(status="active",
limit=100)     mrr = sum(s["items"]["data"][0]["price"]["unit_amount"] * s["items"]
["data"][0]["quantity"] for s in subs["data"]) / 100     return {"mrr": mrr,
"active_subscriptions": len(subs["data"])}  def get_recent_revenue(days: int = 30) ->
dict:     since = int((datetime.now() - timedelta(days=days)).timestamp())
charges = stripe.Charge.list(created={"gte": since}, limit=100)     total =
sum(c["amount"] for c in charges["data"] if c["paid"]) / 100     return
{"total_revenue": total, "transactions": len(charges["data"]), "days": days}
```

23.7 Tool 7 - GitHub API (hermes-builder)

```
pip install PyGithub  # /opt/edvisingu/tools/github_tools.py from github import
Github import os  g = Github(os.environ["GITHUB_TOKEN"])  def create_repo(name: str,
description: str, private: bool = True) -> dict:     repo =
g.get_user().create_repo(name=name, description=description, private=private,
auto_init=True)     return {"url": repo.html_url, "clone": repo.clone_url}  def
push_file(repo_name: str, path: str, content: str, message: str) -> dict:     repo =
g.get_user().get_repo(repo_name)     try:         existing = repo.get_contents(path)
repo.update_file(path, message, content, existing.sha)     except Exception:
repo.create_file(path, message, content)     return {"status": "pushed", "repo":
repo_name, "file": path}
```

23.8 Tool Assignment Per Agent

hermes-core	web_search, create_file, create_notion_page, trigger_workflow
hermes-content	web_search, create_file, trigger_content_pipeline
hermes-advisor	web_search, create_notion_page, append_to_database
hermes-credihire	create_file, web_search
hermes-ops	run_python_code, trigger_workflow, create_file
hermes-social	trigger_workflow, web_search, create_notion_page
hermes-builder	run_python_code, create_repo, push_file, create_file, web_search

hermes-research	web_search, create_file, create_notion_page, run_python_code
hermes-finance	get_mrr, get_recent_revenue, run_python_code, create_file
hermes-email	trigger_email_send, create_notion_page, web_search

SECTION 24: EXPANDED AGENT FLEET - DR. D'S 10-AGENT FOUNDATION FLEET

The original 6-agent blueprint covered EdVisingU education functions. Expanding to 10 gives Dr. D a complete personal AI staff covering all dimensions of the business: executive ops, content, student advising, career tools, community, project building, intelligence, finance, and communications.

24.1 Foundation Fleet - Agents 1-10 (Start Here)

hermes-core (8001)	Executive assistant. Daily planning, strategy, morning briefings, idea capture. Primary agent Dr. D talks to every day.
hermes-content (8002)	Content factory. LinkedIn, TikTok scripts, YouTube outlines, newsletters, podcast topics. Connects to Blotato.
hermes-advisor (8003)	Student advisor. Course recommendations, academic planning, career guidance, HireEd Nexus Lab referrals.
hermes-credihire (8004)	Career agent. ATS scoring, resume rewriting, LinkedIn optimization, cover letters, interview coaching.
hermes-ops (8005)	System monitor. Health checks, backup verification, n8n status, daily morning reports.
hermes-social (8006)	Community. Discord announcements, Whop events, engagement responses, community growth ideas.
hermes-builder (8008)	NEW. Project builder. Creates GitHub repos, scaffolds apps, writes

	boilerplate, plans new product builds.
hermes-research (8009)	NEW. Intelligence. Web search, competitor analysis, market research, trend spotting. Saves reports to /srv/shared-files/research/.
hermes-finance (8010)	NEW. Revenue. Live Stripe MRR, subscription trends, anomaly flags, revenue projections.
hermes-email (8011)	NEW. Communications. Email drafting in Dr. D's voice, Gmail sends via n8n, Notion follow-up tracking, calendar coordination.

24.2 SOUL.md - hermes-builder

```
# SOUL: hermes-builder ## Identity You are Hermes Builder, the product development agent for Dr. Andre De Freitas. ## Mission Build things. Product idea in, working scaffold out. ## Capabilities - Create GitHub repos with proper structure and README - Write Python, JavaScript, TypeScript boilerplate - Design system architecture and database schemas - Write technical specs and SOPs - Scaffold n8n workflows from plain English ## Rules - GitHub repo first, then code - Always include README with setup instructions - Ask if scope is unclear before building - MVP thinking: simplest stack that works - Everything modular and documented
```

24.3 SOUL.md - hermes-research

```
# SOUL: hermes-research ## Identity You are Hermes Research, the intelligence and market analysis agent. ## Mission Find it, analyze it, synthesize it into clear actionable insight. ## Capabilities - Web search via Tavily - Competitor analysis and positioning - EdTech and AI tools trend monitoring - Research reports saved to /srv/shared-files/research/ ## Rules - Always cite sources with URLs - Separate facts from analysis - Lead with the insight, back it with evidence - Flag if info may be outdated - Save all reports as .md files
```

24.4 SOUL.md - hermes-finance

```
# SOUL: hermes-finance ## Identity You are Hermes Finance, the revenue intelligence agent. ## Mission Track money, model scenarios, tell Dr. D exactly where the business stands. ## Capabilities - Live Stripe MRR and subscription data - Revenue trend tracking week over week - Anomaly detection: drops, failed payments, churn - Revenue projections at different growth rates ## Rules - Lead with the number - Flag anything abnormal immediately - Think CFO, not accountant - Recommend pricing changes when data supports it
```

24.5 SOUL.md - hermes-email

```
# SOUL: hermes-email ## Identity You are Hermes Email, the communications agent. ## Mission Handle email drafting, follow-up tracking, and calendar coordination. ## Capabilities - Draft emails in Dr. D's tone: direct, confident, no fluff, no em dashes - Manage follow-ups via Notion - Trigger Gmail sends via n8n - Review calendar and flag conflicts - Draft meeting agendas and post-meeting summaries ## Rules - Always draft first, never send without Dr. D confirmation - Tone: bold, clear, practical - Track all commitments in Notion - Flag urgent emails immediately
```

24.6 Docker Compose for New Agents

```
  hermes-builder:     build: ./agents/hermes-builder     container_name: hermes-builder     restart: always     volumes:       - /srv/hermes-builder/data:/data       - /srv/agent-bus:/agent-bus       - /srv/shared-files:/srv/shared-files     ports: ["8008:8000"]     env_file: /opt/edvisingu/.env    hermes-research:     build: ./agents/hermes-research     container_name: hermes-research     restart: always     volumes:       - /srv/hermes-research/data:/data       - /srv/agent-bus:/agent-bus       - /srv/shared-files:/srv/shared-files     ports: ["8009:8000"]     env_file: /opt/edvisingu/.env    hermes-finance:     build: ./agents/hermes-finance     container_name: hermes-finance     restart: always     volumes:       - /srv/hermes-finance/data:/data       - /srv/agent-bus:/agent-bus       - /srv/shared-files:/srv/shared-files     ports: ["8010:8000"]     env_file: /opt/edvisingu/.env    hermes-email:     build: ./agents/hermes-email     container_name: hermes-email     restart: always     volumes:       - /srv/hermes-email/data:/data       - /srv/agent-bus:/agent-bus       - /srv/shared-files:/srv/shared-files     ports: ["8011:8000"]     env_file: /opt/edvisingu/.env
```

24.7 Redis Task Bus (Production Upgrade)

Replace the file-based agent task bus with Redis for reliable, fast agent-to-agent messaging in production.

```
# docker-compose.yml addition:   redis:     image: redis:7-alpine     container_name: redis     restart: always     ports: ["6379:6379"]     volumes: [redis-data:/data]     command: redis-server --appendonly yes
```

```
pip install redis  # /opt/edvisingu/tools/task_bus.py import redis, json, os  r = redis.from_url(os.environ.get("REDIS_URL", "redis://redis:6379"))  def publish_task(agent: str, action: str, payload: dict) -> dict:     task = {"agent": agent, "action": action, "payload": payload}     r.lpush(f"tasks:{agent}", json.dumps(task))     return {"queued": True, "agent": agent}  def
```

```
get_next_task(agent_name: str):     raw = r.rpop(f"tasks:{agent_name}")     return
json.loads(raw) if raw else None
```

SECTION 25: MONITORING - UPTIME KUMA + TELEGRAM ALERTS

hermes-ops monitors the system. Uptime Kuma is its dashboard. Any service down triggers a Telegram alert to Dr. D within 60 seconds. No manual checking required.

25.1 Uptime Kuma Docker Setup

```
# docker-compose.yml addition:   uptime-kuma:     image: louislam/uptime-kuma:1
container_name: uptime-kuma     restart: always     ports: ["3001:3001"]     volumes:
[uptime-kuma-data:/app/data]
```

Nginx config for status.edvisingu.com:

```
sudo nano /etc/nginx/sites-available/status.edvisingu.com  server {     listen 80;
server_name status.edvisingu.com;     location / {         proxy_pass
http://localhost:3001;         proxy_http_version 1.1;         proxy_set_header
Upgrade $http_upgrade;         proxy_set_header Connection "upgrade";
proxy_set_header Host $host;     } }  sudo ln -s
/etc/nginx/sites-available/status.edvisingu.com /etc/nginx/sites-enabled/ sudo nginx
-t && sudo systemctl reload nginx sudo certbot --nginx -d status.edvisingu.com
```

25.2 What to Monitor

FastAPI Router	HTTP http://localhost:8000/health - every 60s
hermes-core	HTTP http://localhost:8001/health - every 60s
hermes-builder	HTTP http://localhost:8008/health - every 60s
hermes-research	HTTP http://localhost:8009/health - every 60s
hermes-finance	HTTP http://localhost:8010/health - every 60s
n8n	HTTP http://localhost:5678/healthz - every 60s

Open WebUI	HTTP http://localhost:3000 - every 60s
Redis	TCP localhost:6379 - every 30s
ChromaDB	HTTP http://localhost:8800/api/v1/heartbeat - every 60s
agents.edvisingu.com	HTTPS public check - every 60s

25.3 Telegram Alerts Setup

99.

25.4 hermes-ops Daily 7am Briefing (n8n Workflow)

```
# n8n workflow: Daily Morning Status # Node 1: Schedule Trigger - 7:00 AM EST daily (0 12 * * * UTC) # Node 2: HTTP Request POST to http://hermes-ops:8000/chat #   Body: {"message": "Full system health check. Check all containers, n8n workflows, Redis, ChromaDB, Stripe MRR today vs yesterday, any alerts or anomalies. Send morning briefing.", "history": []} # Node 3: Telegram node -> sends response to Dr. D # Result: Morning briefing delivered automatically every day, no request needed
```

25.5 Full Container Fleet on VPS

fastapi-router:8000	Central AI orchestration router
hermes-core:8001	Executive assistant
hermes-content:8002	Content factory
hermes-advisor:8003	Student advisor
hermes-credihire:8004	Resume agent
hermes-ops:8005	System monitor
hermes-social:8006	Community manager

hermes-builder:8008	Project builder (NEW)
hermes-research:8009	Research agent (NEW)
hermes-finance:8010	Revenue tracker (NEW)
hermes-email:8011	Email manager (NEW)
open-webui:3000	Desktop browser interface
hermes-agent (host systemd)	Mobile + messaging gateway. Runs on host, not Docker. Connects Telegram/Discord/Slack to the fleet.
n8n:5678	Automation engine
chromadb:8800	Vector memory
redis:6379	Task bus and caching
uptime-kuma:3001	Monitoring dashboard
nginx:80/443	Reverse proxy with SSL

25.6 VPS RAM Budget

23 agent containers (avg 400MB idle)	9.2 GB
FastAPI router	0.2 GB
Open WebUI	0.5 GB
n8n	0.5 GB
ChromaDB	0.5 GB
Redis	0.1 GB

Nginx	0.1 GB
Uptime Kuma	0.2 GB
Telegram bot	0.2 GB
OS and overhead	1.0 GB
TOTAL	~8.3 GB
CPX41 has	16 GB
Headroom for spikes	7.7 GB - comfortable for burst traffic and agent concurrency

WARNING NEVER run Ollama on the VPS. One 7B model = 4-8GB RAM. It will starve other containers. All inference routes through Claude API and OpenRouter. The VPS is orchestration only, not a model runner.

SECTION 26: MULTI-MODEL ROUTING - RIGHT MODEL FOR EVERY AGENT

The router in Section 22 sends every agent to the same model. This is expensive and slow for simple tasks. The production upgrade assigns each agent to the correct model based on what it does.

26.1 Model Assignment Matrix

Claude Sonnet 4.6	Strategy, writing, advising, research, email, proposals, content. Long-context, nuanced output. Default model.
Claude Haiku 4.5	Speed tasks: ops checks, CRM updates, quick lookups, webhook responses. 10x cheaper than Sonnet.

Gemini 2.0 Flash	Web search with live grounding, Google Drive/Gmail reads, social trend research. Google-native tasks.
GPT-4o via OpenAI	Code generation, debugging, GitHub automation. hermes-builder only.
Gemma 3/4 (Ollama local)	Free open-source model running on VPS. Zero API cost. Used for high-volume simple tasks: formatting, template filling, quick summaries. gemma3:12b via Ollama.
Ollama (fallback)	Offline fallback. Qwen2.5-7B or Phi-3 on VPS CPU when cloud APIs are down or for embeddings.

26.2 Updated FastAPI Router with Model Routing

Replace /opt/edvisingu/fastapi-router/main.py with this version:

```
import os, httpx from fastapi import FastAPI from pydantic import BaseModel from
typing import List, Optional import anthropic import google.generativeai as genai
app = FastAPI()  AGENT_CONTAINERS = {     "hermes-core":         "http://hermes-
core:8000",     "hermes-content":      "http://hermes-content:8000",     "hermes-
advisor":      "http://hermes-advisor:8000",     "hermes-credihire":
"http://hermes-credihire:8000",     "hermes-ops":          "http://hermes-ops:8000",
"hermes-social":       "http://hermes-social:8000",     "hermes-builder":
"http://hermes-builder:8000",     "hermes-research":     "http://hermes-
research:8000",     "hermes-finance":      "http://hermes-finance:8000",     "hermes-
email":        "http://hermes-email:8000",     "hermes-ads":          "http://hermes-
ads:8000",     "hermes-seo":          "http://hermes-seo:8000",     "hermes-funnel":
"http://hermes-funnel:8000",     "hermes-etsy":         "http://hermes-etsy:8000",
"hermes-outreach":     "http://hermes-outreach:8000",     "hermes-proposals":
"http://hermes-proposals:8000",     "hermes-crm":          "http://hermes-crm:8000",
"hermes-crediversity": "http://hermes-crediversity:8000",     "hermes-hireed":
"http://hermes-hireed:8000",     "hermes-educonnect":   "http://hermes-
educonnect:8000",     "hermes-whop":         "http://hermes-whop:8000",     "hermes-
tiktok":       "http://hermes-tiktok:8000",     "hermes-campaign":
"http://hermes-campaign:8000",     "hermes-gumroad":      "http://hermes-
gumroad:8000",     "hermes-pinterest":    "http://hermes-pinterest:8000", }
CODEX_AGENTS = {"hermes-builder"} GEMINI_AGENTS = {"hermes-social", "hermes-seo",
"hermes-tiktok", "hermes-ads", "hermes-pinterest"} HAIKU_AGENTS  = {"hermes-ops",
"hermes-finance", "hermes-crm", "hermes-whop", "hermes-etsy", "hermes-gumroad"}
GEMMA_AGENTS  = set()  # Add agent names here to use free local Gemma3:12b via Ollama
```

```
class Message(BaseModel):     role: str     content: str  class
OAIRequest(BaseModel):     model: str     messages: List[Message]     stream:
Optional[bool] = False  @app.get("/health") def health(): return {"status": "ok",
"agents": len(AGENT_CONTAINERS)}  @app.get("/v1/models") def list_models():
return {"data": [{"id": k, "object": "model"} for k in AGENT_CONTAINERS]}
@app.post("/v1/chat/completions") async def oai_chat(req: OAIRequest):     agent =
req.model if req.model in AGENT_CONTAINERS else "hermes-core"     msg =
req.messages[-1].content     history = [{"role": m.role, "content": m.content} for m
in req.messages[:-1]]     msgs = history + [{"role": "user", "content": msg}]      if
agent in CODEX_AGENTS:         async with httpx.AsyncClient(timeout=120) as c:
r = await c.post("https://api.openai.com/v1/chat/completions",
headers={"Authorization": f"Bearer {os.environ.get("OPENAI_API_KEY","")}"},
json={"model": "gpt-4o", "messages": msgs})         reply = r.json()["choices"][0]
["message"]["content"]     elif agent in GEMINI_AGENTS:
genai.configure(api_key=os.environ["GOOGLE_AI_API_KEY"])         model =
genai.GenerativeModel("gemini-2.0-flash")         result =
model.generate_content(msg)         reply = result.text     elif agent in
HAIKU_AGENTS:         client = anthropic.Anthropic()         r =
client.messages.create(model="claude-haiku-4-5-20251001",
max_tokens=1024, messages=msgs)         reply = r.content[0].text     elif agent in
GEMMA_AGENTS:         # Route to local Gemma3:12b via Ollama (FREE  -  zero API cost)
async with httpx.AsyncClient(timeout=60) as c:             r = await
c.post("http://localhost:11434/api/chat",                 json={"model":
"gemma3:12b", "messages": msgs, "stream": False})         reply =
r.json().get("message", {}).get("content", "Gemma error")     else:         client =
anthropic.Anthropic()         r = client.messages.create(model="claude-sonnet-4-6",
max_tokens=8096, messages=msgs)         reply = r.content[0].text     return {"id":
"jarvis", "object": "chat.completion", "model": agent,         "choices": [{"index":
0, "message": {"role": "assistant", "content": reply}, "finish_reason": "stop"}]}
```

NOTE *Agents not in CODEX_AGENTS, GEMINI_AGENTS, or HAIKU_AGENTS automatically use Claude Sonnet 4.6. Add OPENAI_API_KEY to .env for hermes-builder. GOOGLE_AI_API_KEY for Gemini agents. ANTHROPIC_API_KEY covers all Claude agents.*

26.3 Agent Model Summary

Claude Sonnet 4.6 (default)	hermes-core, hermes-content, hermes-advisor, hermes-credihire, hermes-research, hermes-email, hermes-proposals, hermes-outreach, hermes-crediversity, hermes-hireed, hermes-educonnect, hermes-funnel, hermes-campaign

Claude Haiku 4.5 (fast/cheap)	hermes-ops, hermes-finance, hermes-crm, hermes-whop, hermes-etsy
Gemini 2.0 Flash (web search)	hermes-social, hermes-seo, hermes-tiktok, hermes-ads
GPT-4o Codex (code only)	hermes-builder
Gemma 3/4 via Ollama (free/local)	Any agent added to GEMMA_AGENTS set - zero cost, runs on VPS CPU, no API key needed

SECTION 27: THE FULL 25-AGENT FLEET - NEW AGENTS 11-25

Section 24 covers the first 10 foundation agents. This section covers the 15 specialist agents that complete the 25-agent fleet covering all active revenue channels, platforms, and business functions.

27.1 New Agent Overview (Agents 11-25)

hermes-ads (port 8012)	Facebook/Instagram/TikTok ads copy and targeting. Model: Gemini.
hermes-seo (port 8013)	Keyword research, content briefs, meta descriptions. Model: Gemini.
hermes-funnel (port 8014)	Funnel mapping, conversion copy, ManyChat flow design. Model: Sonnet.
hermes-etsy (port 8015)	Etsy listing copy and tag optimization (SEMI-MANUAL: no direct API). Model: Haiku.
hermes-outreach (port 8016)	Cold email, LinkedIn DMs, partnership pitches. Model: Sonnet.
hermes-proposals (port 8017)	Consulting SOWs, speaking proposals, grant outlines. Model: Sonnet.
hermes-crm (port 8018)	CRM contact updates, lead tracking, follow-up reminders. Model: Haiku.
hermes-crediversity (port 8019)	CrediVersity course build and Disco.co LMS automation. Model: Sonnet.
hermes-hireed (port 8020)	HireEd Nexus Labs project matching and WIL coordination. Model: Sonnet.
hermes-educonnect (port 8021)	EduConnect platform management and advisor workflows. Model: Sonnet.
hermes-whop (port 8022)	Whop membership management, revenue reporting, and Discord

	role automation. Model: Haiku.
hermes-tiktok (port 8023)	TikTok scripts, trending audio, Blotato scheduling. Model: Gemini.
hermes-campaign (port 8024)	DrDDurham political campaign content and Blotato publishing. Model: Sonnet.
hermes-gumroad (port 8025)	Gumroad product listings, pricing, sales page copy, and discount codes. Model: Haiku.
hermes-skillplate (port 8026)	Skillplate.com owned storefront - product listings, pricing updates, order tracking, and coupon management. No marketplace fees or algorithm dependency. Model: Haiku.
hermes-pinterest (port 8027)	Pinterest pin creation, board strategy, SEO descriptions, and Blotato scheduling to drive traffic to Etsy, Gumroad, and Skillplate. Model: Gemini.

27.2 SOUL.md Files for New Agents

hermes-ads

```
# SOUL.md - hermes-ads You are hermes-ads, Dr. D's paid advertising strategist. Capabilities: Facebook/Instagram/TikTok ad copy, audience targeting, campaign structure, A/B test suggestions, ROAS analysis. Rules: Never recommend spending without Dr. D approval. Every ad needs hook + benefit + CTA. Report ROAS and CPC weekly. Integrations: Meta Ads Manager (manual review), TikTok Ads Manager (manual review), Blotato for organic. Model: Gemini 2.0 Flash (live ad performance benchmarks via web grounding).
```

hermes-seo

```
# SOUL.md - hermes-seo You are hermes-seo, Dr. D's SEO and content research specialist. Capabilities: keyword research, content briefs, meta tags, internal linking, competitor gap analysis. Rules: All keyword output must include search volume estimate and intent (informational/commercial/transactional). Primary sites: edvisingu.com, crediversity.com, gohireed.com, ai.edvisingu.com. Model: Gemini 2.0 Flash (live SERP data via web grounding).
```

hermes-funnel

```
# SOUL.md - hermes-funnel You are hermes-funnel, Dr. D's conversion and funnel specialist. Capabilities: funnel mapping, offer creation, landing page copy, email sequence strategy, ManyChat flow design, upsell/downsell sequencing. Rules: Every funnel recommendation must include entry point, lead magnet, tripwire, core offer, and upsell. ManyChat role: Design Instagram DM automations (comment triggers, keyword replies, story mention flows) that capture leads and push them into MailerLite sequences. ManyChat is the top-of-funnel capture layer for organic social traffic. ManyChat + n8n integration: ManyChat webhook fires when a subscriber opts in -> n8n receives -> tags subscriber in MailerLite -> logs to Supabase -> triggers welcome sequence. Integrations: ManyChat (Instagram/FB DM automation), MailerLite (email sequences), Whop (membership checkout), Stripe (payment), ChatbotBuilder.net (student-facing flows). Model: Claude Sonnet 4.6.
```

hermes-etsy

```
# SOUL.md - hermes-etsy You are hermes-etsy, Dr. D's Etsy shop assistant. Capabilities: Etsy listing copy, title optimization, tag selection (13 tags max), pricing strategy. CRITICAL LIMITATION: Etsy API requires PKCE authentication. n8n CANNOT automate Etsy posting. All publishing is SEMI-MANUAL. Workflow: Generate optimized listing content -> Dr. D copy-pastes into Etsy Seller Hub. Never attempt direct API posting. Model: Claude Haiku 4.5 (fast lookups, low cost).
```

hermes-outreach

```
# SOUL.md - hermes-outreach You are hermes-outreach, Dr. D's outbound relationship builder. Capabilities: cold email drafting, LinkedIn DM scripts, partnership pitch emails, podcast guest pitches. Rules: Every outreach must be personalized. Research the recipient via Tavily first. No generic templates. Tone: peer-to-peer, value-first. Integrations: Gmail API, MailerLite, Notion (log all outreach attempts and responses). Model: Claude Sonnet 4.6.
```

hermes-proposals

```
# SOUL.md - hermes-proposals You are hermes-proposals, Dr. D's consulting and partnership proposal writer. Capabilities: consulting SOW drafts, speaking engagement proposals, institutional partnership decks, grant outlines. Rules: All proposals must include: problem statement, Dr. D's unique value, deliverables, timeline, investment ask, and ROI case. Output: Google Doc via Drive API, DOCX, or PDF. Tone: executive, authoritative, results-focused. Model: Claude Sonnet 4.6.
```

hermes-crm

```
# SOUL.md - hermes-crm You are hermes-crm, Dr. D's lightweight CRM assistant. Capabilities: contact record updates in Notion, lead status tracking, follow-up
```

```
reminders, pipeline health reports. Rules: Every contact interaction must be logged in Notion. Follow-up tasks auto-created if no response in 5 days. Integrations: Notion API, Gmail API (read threads), Telegram alerts for hot leads. Model: Claude Haiku 4.5 (fast record updates).
```

hermes-crediversity

```
# SOUL.md - hermes-crediversity You are hermes-crediversity, the AI builder for CrediVersity. Capabilities: micro-credential design, Disco.co course setup, module structuring, assessment design, learner outcome writing, badge specification. Rules: All credentials must align to real workforce skill gaps. Every module must have a measurable outcome. No filler content. Integrations: Disco.co API, Google Drive (course materials), Stripe (payment links). Model: Claude Sonnet 4.6.
```

hermes-hireed

```
# SOUL.md - hermes-hireed You are hermes-hireed, the HireEd Nexus Labs project coordinator. Capabilities: project brief creation, student-employer matching, WIL setup, milestone tracking, deliverable reviews. Platform: gohireed.com - HireEd Nexus Labs. Rules: All projects must have defined scope, student outcomes, employer deliverables, and payment structure. No ambiguous scopes. Integrations: Notion (project boards), Google Calendar (milestones), Stripe (payment collection). Model: Claude Sonnet 4.6.
```

hermes-educonnect

```
# SOUL.md - hermes-educonnect You are hermes-educonnect, the EduConnect platform coordinator. Capabilities: student advisor workflow management, EduConnect platform updates, advisor assignment, OSAP/BSWD automation support. Platform: ai.edvisingu.com - EduConnect. Rules: Protect student privacy. FIPPA and FERPA compliant data handling. No student data stored outside Supabase. Integrations: Supabase (student records), Notion (advisor notes), n8n (workflow triggers). Model: Claude Sonnet 4.6.
```

hermes-whop

```
# SOUL.md - hermes-whop You are hermes-whop, Dr. D's Whop membership operations manager. Capabilities: product creation, pricing updates, discount codes, member access management, webhook event processing, MRR reporting. Rules: Always verify Whop webhook signatures before processing. Report MRR to Dr. D every Monday. Integrations: Whop API (api.whop.com), Stripe, Telegram (notify Dr. D on new member joins). Model: Claude Haiku 4.5.
```

hermes-tiktok

```
# SOUL.md - hermes-tiktok You are hermes-tiktok, Dr. D's TikTok content specialist. Capabilities: 60-90 second script writing, trending audio identification, caption + hashtag generation, content calendar building, Blotato scheduling. Rules: Every script must start with a pattern interrupt hook in the first 2 seconds. End with clear CTA. Target posting cadence: 4-5x per week. Integrations: Blotato API (schedule posts), Google Drive (store scripts). Model: Gemini 2.0 Flash (live TikTok trends via web grounding).
```

hermes-campaign

```
# SOUL.md - hermes-campaign You are hermes-campaign, the AI campaign manager for DrDDurham (Pickering election, October 26, 2026). Capabilities: Facebook/Instagram/TikTok content, X posts, volunteer coordination emails, event announcements, policy copy, fundraising appeals. Brand: DrDDurham  -  Navy #1B3A5C + Gold #D4A843. Slogans: Built to Lead. Ready for Pickering. Rules: All content must comply with Canadian electoral law and the Ontario Municipal Elections Act. No misinformation. All paid ads need authorization tagline. Integrations: Blotato (schedule), MailerLite (campaign@drddurham.ca), Gmail API (volunteer emails). Model: Claude Sonnet 4.6.
```

hermes-gumroad

```
# SOUL.md - hermes-gumroad You are hermes-gumroad, Dr. D's Gumroad digital product manager. Capabilities: Gumroad product page copy, pricing optimization, sales description writing, discount code strategy, upsell sequencing, product bundle ideas, sales analytics reporting. Platform: gumroad.com (Dr. D's Gumroad store for digital products - guides, templates, courses). Rules: Every product page must have a clear headline, benefit bullets, social proof placeholder, and strong CTA. Pricing must align with Whop and Etsy strategy to avoid channel conflict. Integrations: Gumroad API, Stripe (revenue tracking), MailerLite (buyer follow-up sequences), Notion (product inventory log). Model: Claude Haiku 4.5.
```

hermes-pinterest

```
# SOUL.md - hermes-pinterest You are hermes-pinterest, Dr. D's Pinterest content strategist. Primary goal: Drive traffic to Etsy listings and Gumroad products via Pinterest SEO and viral pins. Capabilities: pin title and description writing (keyword-optimized for Pinterest search), board naming and strategy, pin image prompt generation (for Canva/Midjourney), weekly pin scheduling via Blotato, Pinterest analytics interpretation, cross-linking to Etsy and Gumroad product URLs. Rules: Every pin must include a product URL (Etsy or Gumroad). Descriptions must be 100-200 chars with 3-5 relevant keywords. Post 10-15 pins per week minimum. Focus boards: Education, AI Tools, Student Success, Digital Products, Online Income. Integrations: Blotato API (schedule pins), Etsy (product URLs), Gumroad (product URLs), Canva (image generation prompts stored in Google Drive). Model: Gemini 2.0 Flash (live Pinterest trend data via web grounding).
```

27.3 Docker Compose - New Agents 11-25

Add these service blocks after hermes-email in /opt/edvisingu/docker-compose.yml:

```
  hermes-ads:     build: ./agents/hermes-ads     container_name: hermes-ads
restart: always     volumes:       - /srv/hermes-ads/data:/data       - /srv/shared-
files:/srv/shared-files     ports: ["8012:8000"]     env_file: /opt/edvisingu/.env
hermes-seo:     build: ./agents/hermes-seo     container_name: hermes-seo
restart: always     volumes:       - /srv/hermes-seo/data:/data       - /srv/shared-
files:/srv/shared-files     ports: ["8013:8000"]     env_file: /opt/edvisingu/.env
hermes-funnel:     build: ./agents/hermes-funnel     container_name: hermes-funnel
restart: always     ports: ["8014:8000"]     env_file: /opt/edvisingu/.env    hermes-
etsy:     build: ./agents/hermes-etsy     container_name: hermes-etsy     restart:
always     ports: ["8015:8000"]     env_file: /opt/edvisingu/.env    hermes-outreach:
build: ./agents/hermes-outreach     container_name: hermes-outreach     restart:
always     ports: ["8016:8000"]     env_file: /opt/edvisingu/.env    hermes-
proposals:     build: ./agents/hermes-proposals     container_name: hermes-proposals
restart: always     ports: ["8017:8000"]     env_file: /opt/edvisingu/.env    hermes-
crm:     build: ./agents/hermes-crm     container_name: hermes-crm     restart:
always     ports: ["8018:8000"]     env_file: /opt/edvisingu/.env    hermes-
crediversity:     build: ./agents/hermes-crediversity     container_name: hermes-
crediversity     restart: always     ports: ["8019:8000"]     env_file:
/opt/edvisingu/.env    hermes-hireed:     build: ./agents/hermes-hireed
container_name: hermes-hireed     restart: always     ports: ["8020:8000"]
env_file: /opt/edvisingu/.env    hermes-educonnect:     build: ./agents/hermes-
educonnect     container_name: hermes-educonnect     restart: always     ports:
["8021:8000"]     env_file: /opt/edvisingu/.env    hermes-whop:     build:
./agents/hermes-whop     container_name: hermes-whop     restart: always     ports:
["8022:8000"]     env_file: /opt/edvisingu/.env    hermes-tiktok:     build:
./agents/hermes-tiktok     container_name: hermes-tiktok     restart: always
ports: ["8023:8000"]     env_file: /opt/edvisingu/.env    hermes-campaign:     build:
./agents/hermes-campaign     container_name: hermes-campaign     restart: always
ports: ["8024:8000"]     env_file: /opt/edvisingu/.env    hermes-gumroad:
build: ./agents/hermes-gumroad     container_name: hermes-gumroad     restart: always
ports: ["8025:8000"]     env_file: /opt/edvisingu/.env    hermes-pinterest:
build: ./agents/hermes-pinterest     container_name: hermes-pinterest     restart:
always     ports: ["8026:8000"]     env_file: /opt/edvisingu/.env
```

NOTE *Total VPS container count with all 25 agents + infrastructure: 34 containers. CPX41 handles this comfortably with 16GB RAM. Build agents in batches of 5. Test each batch before adding the next.*

SECTION 28: GOOGLE WORKSPACE FULL API INTEGRATION

Dr. D has an active Google Workspace subscription. This section covers full API integration so agents can read Gmail, create Calendar events, manage Google Drive files, and use Gemini with live web grounding.

28.1 Google Cloud Project Setup

100. Go to https://console.cloud.google.com and sign in with Dr. D's Google Workspace account

101. Create a new project named: edvisingu-jarvis

102. Enable these APIs: Gmail API, Google Calendar API, Google Drive API, Google Docs API, Google Generative Language API

103. Create OAuth 2.0 credentials (Desktop Application type for first token generation)

104. Download credentials.json - store in Bitwarden, NEVER commit to GitHub

105. Run one-time OAuth flow on your local machine to generate refresh_token

106. Add refresh_token to GOOGLE_REFRESH_TOKEN in master .env on VPS

107. Also get a GOOGLE_AI_API_KEY from https://aistudio.google.com (separate from OAuth credentials)

28.2 Gmail API Tool

```
# /opt/edvisingu/tools/gmail_tools.py import os, base64 from
google.oauth2.credentials import Credentials from google.auth.transport.requests
import Request from googleapiclient.discovery import build from email.mime.text
import MIMEText  def get_gmail_service():     creds =
Credentials(         token=None,
refresh_token=os.environ["GOOGLE_REFRESH_TOKEN"],
client_id=os.environ["GOOGLE_CLIENT_ID"],
client_secret=os.environ["GOOGLE_CLIENT_SECRET"],
token_uri="https://oauth2.googleapis.com/token",
scopes=["https://www.googleapis.com/auth/gmail.modify"]     )
creds.refresh(Request())     return build("gmail", "v1", credentials=creds)  def
get_recent_emails(max_results=10):     service = get_gmail_service()     results =
service.users().messages().list(userId="me", maxResults=max_results,
labelIds=["INBOX"]).execute()     output = []     for msg in results.get("messages",
[]):         detail = service.users().messages().get(userId="me", id=msg["id"],
format="metadata").execute()         headers = {h["name"]: h["value"] for h in
```

```
detail["payload"]["headers"]}         output.append({"from": headers.get("From",""),
"subject": headers.get("Subject","")})     return output  def send_email(to: str,
subject: str, body: str):     service = get_gmail_service()     message =
MIMEText(body)     message["to"] = to     message["subject"] = subject     raw =
base64.urlsafe_b64encode(message.as_bytes()).decode()
service.users().messages().send(userId="me", body={"raw": raw}).execute()
```

28.3 Google Calendar API Tool

```
# /opt/edvisingu/tools/calendar_tools.py import os from google.oauth2.credentials
import Credentials from google.auth.transport.requests import Request from
googleapiclient.discovery import build from datetime import datetime, timedelta  def
get_calendar_service():     creds = Credentials(         token=None,
refresh_token=os.environ["GOOGLE_REFRESH_TOKEN"],
client_id=os.environ["GOOGLE_CLIENT_ID"],
client_secret=os.environ["GOOGLE_CLIENT_SECRET"],
token_uri="https://oauth2.googleapis.com/token",
scopes=["https://www.googleapis.com/auth/calendar"]     )
creds.refresh(Request())     return build("calendar", "v3", credentials=creds)  def
get_todays_events():     service = get_calendar_service()     now =
datetime.utcnow().isoformat() + "Z"     end = (datetime.utcnow() +
timedelta(days=1)).isoformat() + "Z"     events =
service.events().list(calendarId="primary", timeMin=now, timeMax=end,
singleEvents=True, orderBy="startTime").execute()     return events.get("items", [])
def create_event(title: str, start: str, end: str, description: str = ""):
service = get_calendar_service()     event = {"summary": title, "description":
description,              "start": {"dateTime": start, "timeZone":
"America/Toronto"},              "end": {"dateTime": end, "timeZone":
"America/Toronto"}}     return service.events().insert(calendarId="primary",
body=event).execute()
```

28.4 Gemini API with Web Grounding

```
# /opt/edvisingu/tools/gemini_tools.py import os import google.generativeai as genai
def gemini_search(query: str) -> str:
genai.configure(api_key=os.environ["GOOGLE_AI_API_KEY"])     model =
genai.GenerativeModel("gemini-2.0-flash")     result = model.generate_content(
query,         tools=[{"google_search": {}}]  # Live web grounding     )     return
result.text  def gemini_analyze(content: str, prompt: str) -> str:
genai.configure(api_key=os.environ["GOOGLE_AI_API_KEY"])     model =
genai.GenerativeModel("gemini-2.0-flash")     result =
model.generate_content(f"{prompt}\n\nContent:\n{content}")     return result.text
```

> **NOTE** *Use gemini_search() in hermes-seo, hermes-ads, hermes-tiktok, and hermes-social for all live*

> *trend and research queries. Gemini is natively integrated with Google Search - it sees real web data at the time of the call.*

28.5 Google Drive API Tool

```
# /opt/edvisingu/tools/drive_tools.py import os from google.oauth2.credentials import
Credentials from google.auth.transport.requests import Request from
googleapiclient.discovery import build  def get_drive_service():     creds =
Credentials(         token=None,
refresh_token=os.environ["GOOGLE_REFRESH_TOKEN"],
client_id=os.environ["GOOGLE_CLIENT_ID"],
client_secret=os.environ["GOOGLE_CLIENT_SECRET"],
token_uri="https://oauth2.googleapis.com/token",
scopes=["https://www.googleapis.com/auth/drive"]     )     creds.refresh(Request())
return build("drive", "v3", credentials=creds)  def
list_recent_files(max_results=20):     service = get_drive_service()     results =
service.files().list(pageSize=max_results,         orderBy="modifiedTime desc",
fields="files(id, name, mimeType, modifiedTime)").execute()     return
results.get("files", [])  def search_files(query: str):     service =
get_drive_service()     results = service.files().list(q=f"fullText contains
'{query}'",         fields="files(id, name, mimeType)").execute()     return
results.get("files", [])
```

SECTION 29: PLATFORM INTEGRATIONS PART 2 - BLOTATO, MAILERLITE, MANYCHAT, DISCO.CO

29.1 Blotato - Social Media Scheduling

Blotato is Dr. D's social media scheduler (replaced Buffer). Handles Facebook, Instagram, TikTok, X, LinkedIn, YouTube from one API call.

Used by	hermes-content, hermes-social, hermes-tiktok, hermes-campaign
API Key	BLOTATO_API_KEY in master .env (get from Bitwarden)
Docs	https://docs.blotato.com/api
Key feature	Multi-platform publish in one call, content calendar management

```
# /opt/edvisingu/tools/blotato_tools.py import os, httpx  BLOTATO_BASE =
"https://api.blotato.com/v1"  async def schedule_post(content: str, platforms: list,
scheduled_at: str) -> dict:     async with httpx.AsyncClient() as client:         r =
await client.post(f"{BLOTATO_BASE}/posts",             headers={"Authorization":
f"Bearer {os.environ['BLOTATO_API_KEY']}"},             json={"content": content,
"platforms": platforms, "scheduled_at": scheduled_at})     return r.json()  async def
get_recent_posts(limit: int = 10) -> dict:     async with httpx.AsyncClient() as
client:         r = await client.get(f"{BLOTATO_BASE}/posts?limit={limit}",
headers={"Authorization": f"Bearer {os.environ['BLOTATO_API_KEY']}"}         )
return r.json()
```

29.2 MailerLite - Email Platform

MailerLite hosts Dr. D's email lists and sequences. 5 active sequences, 22 email bodies. Managed by Lahari. hermes-email reads list data and drafts content. Lahari reviews before sending.

API Key	MAILERLITE_API_KEY in master .env

Sequence owner	Lahari (Dr. D approves all sends)
Active data	5 sequences, 22 email bodies across EdVisingU + DrDDurham lists
Docs	https://developers.mailerlite.com/docs/

```
# /opt/edvisingu/tools/mailerlite_tools.py import os, httpx  ML_BASE =
"https://connect.mailerlite.com/api"  async def get_subscriber_count() -> int:
async with httpx.AsyncClient() as client:         r = await
client.get(f"{ML_BASE}/subscribers?limit=1",             headers={"Authorization":
f"Bearer {os.environ['MAILERLITE_API_KEY']}", "Content-Type": "application/json"})
return r.json().get("meta", {}).get("total", 0)  async def get_campaigns() -> list:
async with httpx.AsyncClient() as client:         r = await
client.get(f"{ML_BASE}/campaigns",             headers={"Authorization": f"Bearer
{os.environ['MAILERLITE_API_KEY']}"})     return r.json().get("data", [])  async def
add_subscriber(email: str, name: str = "") -> dict:     async with
httpx.AsyncClient() as client:         r = await
client.post(f"{ML_BASE}/subscribers",             headers={"Authorization": f"Bearer
{os.environ['MAILERLITE_API_KEY']}"},             json={"email": email, "fields":
{"name": name}})     return r.json()
```

WARNING hermes-email can READ MailerLite data and draft email content. It should NOT send campaigns automatically without Dr. D's explicit approval. All sends go through Lahari for QA first.

29.3 ManyChat - Messenger and Instagram DM Automation

ManyChat handles Facebook Messenger and Instagram DM funnels. Connect to n8n via webhook to trigger lead capture and funnel events.

```
# n8n webhook setup for ManyChat: # 1. ManyChat > Settings > Integrations > Webhooks
# 2. Add URL: https://n8n.edvisingu.com/webhook/manychat-lead # 3. Select triggers:
New Subscriber, Tag Applied, Sequence Completed # 4. n8n Webhook node at /manychat-
lead routes to hermes-funnel or hermes-crm  # /opt/edvisingu/tools/manychat_tools.py
import os, httpx  MC_BASE = "https://api.manychat.com"  async def
get_subscriber_info(subscriber_id: str) -> dict:     async with httpx.AsyncClient()
as client:         r = await client.get(f"{MC_BASE}/fb/subscriber/getInfo?
subscriber_id={subscriber_id}",             headers={"Authorization": f"Bearer
{os.environ['MANYCHAT_API_KEY']}"})     return r.json()  async def
tag_subscriber(subscriber_id: str, tag_id: str) -> dict:     async with
httpx.AsyncClient() as client:         r = await
```

```
client.post(f"{MC_BASE}/fb/subscriber/addTag",             headers={"Authorization":
f"Bearer {os.environ['MANYCHAT_API_KEY']}"},            json={"subscriber_id":
subscriber_id, "tag_id": tag_id})     return r.json()
```

29.4 Disco.co - CrediVersity LMS

Disco.co is the learning management system for CrediVersity micro-credentials. hermes-crediversity uses the Disco API for course management and enrollment.

```
# /opt/edvisingu/tools/disco_tools.py import os, httpx  DISCO_BASE =
"https://api.disco.co/v1"  async def get_courses() -> list:     async with
httpx.AsyncClient() as client:         r = await client.get(f"{DISCO_BASE}/courses",
headers={"Authorization": f"Bearer {os.environ['DISCO_API_KEY']}"})     return
r.json().get("data", [])  async def get_enrollments(course_id: str) -> list:
async with httpx.AsyncClient() as client:         r = await
client.get(f"{DISCO_BASE}/courses/{course_id}/enrollments",
headers={"Authorization": f"Bearer {os.environ['DISCO_API_KEY']}"})     return
r.json().get("data", [])  async def enroll_learner(course_id: str, email: str) ->
dict:     async with httpx.AsyncClient() as client:         r = await
client.post(f"{DISCO_BASE}/courses/{course_id}/enrollments",
headers={"Authorization": f"Bearer {os.environ['DISCO_API_KEY']}"},
json={"email": email})     return r.json()
```

SECTION 30: DrDDURHAM POLITICAL CAMPAIGN - OCTOBER 2026 PICKERING ELECTION

Dr. D is running for elected office in Pickering, Ontario (October 26, 2026). hermes-campaign manages the full AI-powered content and automation stack for this campaign.

30.1 Campaign Brand and Platform Specs

Campaign name	DrDDurham
Handle	@DrDDurham (all platforms - claim immediately)
Primary slogan	Built to Lead. Ready for Pickering.
Brand colors	Navy #1B3A5C + Gold #D4A843 + White
Campaign email	campaign@drddurham.ca
Website	drddurham.ca (Vercel landing page)
Election date	October 26, 2026 (Pickering, Ontario)
Primary platform	Facebook Page @DrDDurham
Secondary platforms	Instagram, TikTok, X (Twitter), YouTube

30.2 Campaign Tech Stack

- Blotato: schedule content across Facebook, Instagram, TikTok, X from one dashboard
- MailerLite: campaign@drddurham.ca subscriber list (volunteer coordination, GOTV emails)
- Higgsfield: AI video generation for campaign ads and talking-head content
- Canva: graphics, carousels, lawn sign design, branded templates
- Claude (hermes-campaign): write all scripts, captions, policy copy, fundraising appeals
- n8n: connect Claude + Blotato + Higgsfield + MailerLite into automated weekly pipeline

30.3 Weekly Campaign Content Automation

108. Every Sunday: n8n triggers hermes-campaign with the week's events and local Pickering news (via Tavily)

109. hermes-campaign generates: 7 Facebook posts, 5 TikTok scripts, 3 Instagram carousels, 2 email drafts

110. All content saved to Google Drive campaign folder (drive.google.com) for Dr. D review

111. Dr. D reviews and approves or revises via Telegram: /approve-campaign or /revise [note]

112. On approval: hermes-campaign calls Blotato API to schedule all approved posts for the week

30.4 Canadian Electoral Compliance Rules

> **WARNING** All AI-generated campaign content must comply with the Canada Elections Act and Ontario Municipal Elections Act (2024 amendments). Non-compliance is a legal liability.

- All paid ads must include: "Authorized by [Campaign Agent Name], [Campaign Name]"
- Track all campaign expenses (including AI tool costs) in a separate spreadsheet - required by law
- No vote-buying, voter suppression, or misleading statements about voting procedures
- Check Elections Ontario (elections.on.ca) for exact ad blackout periods before scheduling
- AI content assistance is legal under current Canadian law (May 2026); stay updated

SECTION 31: AGENT CREATION FRAMEWORK - ADD ANY NEW AGENT IN 4 STEPS

The Jarvis system is designed to scale infinitely. Every new agent follows the same 4-step pattern. A new specialist agent can be deployed in under 2 hours.

31.1 The 4-Step Pattern

Step 1	Write SOUL.md - the agent's identity, mission, capabilities, rules, tools, model
Step 2	Write main.py - FastAPI server with /health and /chat endpoints and tool imports
Step 3	Write Dockerfile - Python 3.11-slim, copy SOUL.md, install deps, uvicorn CMD
Step 4	Register agent - one line in AGENT_CONTAINERS dict + model group assignment

31.2 Template: SOUL.md

```
# SOUL.md - hermes-[name] You are hermes-[name], Dr. D's [role]. Capabilities: [3-5 specific tasks this agent does] Rules: [2-3 hard constraints - what it never does] Integrations: [APIs and tools this agent uses] Memory: [what it stores in ChromaDB or Supabase] Model: [Claude Sonnet / Claude Haiku / Gemini / GPT-4o] Owner: Dr. Andre De Freitas (andre@edvisingu.ca)
```

31.3 Template: main.py

```
import os from fastapi import FastAPI from pydantic import BaseModel from typing import List app = FastAPI() with open("SOUL.md") as f: SOUL = f.read() class ChatRequest(BaseModel): message: str history: List[dict] = [] @app.get("/health") def health(): return {"status": "ok", "agent": "hermes-[name]"} @app.post("/chat") async def chat(req: ChatRequest): # FastAPI router handles model selection and AI call # This endpoint handles tool use and state management return {"response": f"hermes-[name] received: {req.message}"}
```

31.4 Template: Dockerfile

```
FROM python:3.11-slim WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY . . EXPOSE 8000 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

31.5 Register in Router and Docker Compose

In fastapi-router/main.py, add to AGENT_CONTAINERS:

```
"hermes-[name]": "http://hermes-[name]:8000",
```

Assign model (add to the correct set if not using default Sonnet):

```
# Haiku (fast/cheap): HAIKU_AGENTS.add("hermes-[name]") # Gemini (web search): GEMINI_AGENTS.add("hermes-[name]") # Codex (code only):  CODEX_AGENTS.add("hermes-[name]") # Sonnet (default):   no change needed
```

In docker-compose.yml, add service block:

```
  hermes-[name]:     build: ./agents/hermes-[name]     container_name: hermes-[name] restart: always     ports: ["80XX:8000"]  # next available port after 8024 env_file: /opt/edvisingu/.env
```

31.6 Deploy and Test

```
# Build and start the new agent docker compose build hermes-[name] docker compose up -d hermes-[name]  # Test health curl http://localhost:80XX/health  # Test chat curl -X POST http://localhost:80XX/chat \   -H "Content-Type: application/json" \   -d '{"message":"hello","history":[]}'  # Test via FastAPI router (OpenAI-compatible) curl -X POST https://agents.edvisingu.com/v1/chat/completions \   -H "Content-Type: application/json" \   -d '{"model":"hermes-[name]","messages":[{"role":"user","content":"hello"}]}'  # Add to Uptime Kuma monitoring # URL: http://hermes-[name]:8000/health  Type: HTTP(s)  Interval: 60s
```

APPENDIX A: GLOSSARY

Blotato	AI-native social media scheduler and repurposing tool - publishes across TikTok, LinkedIn, Instagram, YouTube
ChromaDB	Vector database for AI memory - stores content as mathematical embeddings for semantic search
Claude	Anthropic AI model - primary reasoning and writing engine for this ecosystem
Discord	Community platform - Dr. D owns the server; bot automates role assignment, announcements, and onboarding
FastAPI	Python web framework - the specialist agent router (port 8000). Works under OpenJarvis orchestration, receives dispatched subtasks and routes to the correct specialist agent container.
Hermes Agent	Dr. D's custom persistent AI assistant built with LangChain + Claude + ChromaDB
HireEd Nexus Labs	Platform connecting students to real work-integrated projects at gohireed.com
LangChain	Python framework for building AI agents with memory, tools, and retrieval
n8n	Workflow automation platform - the orchestration backbone connecting all systems
Ollama	Runs language models locally on CPU without internet dependency
OpenRouter	Single API that routes to multiple AI models (Claude, GPT, Gemini, Mistral, etc.)
pgvector	Postgres extension in Supabase for vector embedding storage and similarity search
RAG	Retrieval-Augmented Generation - AI retrieves stored knowledge before generating responses

Riverside.fm	AI-powered podcast and video recording platform with built-in transcription and editing
Supabase	Database + auth + storage + real-time - backend for the entire ecosystem
Tailscale	Secure private VPN for remote development access to Dr. D's machine
Vercel	Hosting platform for Next.js dashboards - auto-deploys from GitHub
Whop	Membership and monetization platform - handles $47/month recurring memberships and digital product sales
WSL2	Windows Subsystem for Linux - Not used in this project. Hetzner VPS provides the Linux environment. Students connect via SSH.

APPENDIX B: KEY RESOURCES AND URLS

EdVisingU	https://edvisingu.com
CrediVersity	https://crediversity.com
AI Hub	https://ai.edvisingu.com
HireEd Nexus Labs	https://gohireed.com
Ollama	https://ollama.com
LangChain Python Docs	https://python.langchain.com/docs
FastAPI Docs	https://fastapi.tiangolo.com
Supabase Docs	https://supabase.com/docs
n8n Docs	https://docs.n8n.io
ChromaDB Docs	https://docs.trychroma.com

Anthropic Claude API	https://docs.anthropic.com
OpenRouter	https://openrouter.ai
Vercel Docs	https://vercel.com/docs
Tailscale KB	https://tailscale.com/kb
Supabase pgvector Guide	https://supabase.com/docs/guides/ai/vector-columns
HeyGen API Docs	https://docs.heygen.com
ElevenLabs Python SDK	https://github.com/elevenlabs/elevenlabs-python
Blotato	https://blotato.com
Riverside.fm	https://riverside.fm
Discord Developer Portal	https://discord.com/developers/applications
Bitwarden (free)	https://bitwarden.com
Notion (free)	https://notion.so
Google AI Studio	https://aistudio.google.com

APPENDIX C: CRITICAL WARNINGS SUMMARY

WARNING OpenJarvis (open-jarvis/OpenJarvis) = the TOP-LEVEL AI OS for this project. It is NOT a home automation framework. Install it on Hetzner via: curl -fsSL https://openjarvis.ai/install.sh | bash

WARNING hermes-agent (NousResearch/hermes-agent) IS real software (153k stars). Use it as the gateway layer. Do NOT build a custom Telegram bot - hermes-agent gateway replaces it entirely.

WARNING No GPU/CUDA on the HP EliteBook. Keep Ollama models at 7B parameters or less - larger models will be unusably slow on CPU.

WARNING Never commit .env files or API keys to GitHub. Rotate any key that gets accidentally exposed - immediately.

WARNING Always message Dr. D before connecting remotely. Never access outside approved hours or touch personal files.

WARNING This document is CONFIDENTIAL. Do not share outside the authorized project team.

-- END OF DOCUMENT --

EdVisingU | Dr. Andre De Freitas | andre@edvisingu.ca